

SDC File

Tuesday, September 15, 2026 12:37 PM

What is an .sdc File?

Define Timing. Guide Synthesis. Enable a Successful Chip.

1 What is an .sdc file?

- An **.sdc** file is a text file that specifies the timing requirements and design constraints for an ASIC design.
- It tells the EDA tools (e.g., Genus, Innovus, Tempus) what to achieve and what to verify.
- Without an **.sdc** file, the tools do not know the required timing targets, I/O environment or which paths to ignore.

“SDC is like a specification document for the tools – it defines the rules of the game for timing.”

2 What is the full form of SDC?

- SDC stands for **Synopsys Design Constraints**.
- It was developed by Synopsys in the early 1990s for their Design Compiler and has become an industry standard.
- Even though other EDA tools (e.g., Cadence) also support it, the name remains SDC.

SYNOPSYS®
Developed in 1990s for Design Compiler

3 Why is SDC required in ASIC design?

- Provides timing targets:** Specifies clock frequency and timing goals.
- Defines the design environment:** Models input/output delays and external interfaces.
- Guides optimization:** Helps synthesis and PnR tools make timing-driven decisions.
- Prevents wasted effort:** Identifies false paths and multicycle paths.
- Essential for timing signoff:** Used by STA to verify that the design meets all timing requirements.

4 What type of file is .sdc?

- SDC is a **plain text file**.
- It is written in **TCL (Tool Command Language)** format.
- It contains commands and values understandable by EDA tools.
- File extension is **.sdc** (e.g., design.sdc).

Example: design.sdc

```
# Clock definition
create_clock -period 1.0 [get_ports clk]

# Input delay
set_input_delay -max 0.3 [get_ports {*_in}]

# False path
set_false_path -from [get_ports {*_in}]
```

5 Why is SDC TCL-based?

- EDA tools from Synopsys use TCL as their command language, so SDC is written in TCL.
- TCL is a powerful, flexible, and easy-to-use scripting language.
- It allows you to write readable constraints, use variables, loops, and conditionals if needed.
- Using TCL provides consistency across different tools and flows (Synthesis, PnR, STA).

“TCL makes SDC flexible, scalable, and easy to maintain.”

6 What information does an SDC file contain?

- An SDC file contains all the timing and design rule constraints, including:
 - Clock constraints** : create_clock, create_generated_clock, clock uncertainty, latency
 - I/O constraints** : set_input_delay, set_output_delay, virtual clocks
 - Timing exceptions** : set_false_path, set_multicycle_path, set_max_delay, set_min_delay
 - Design rule constraints** : set_max_transition, set_max_capacitance, set_max_fanout, set_load
 - Mode constraints** : set_case_analysis (e.g., for scan/low-power)
 - Others** : clock groups, propagated clocks, driving cells, etc.

Right Constraints → Right Optimization → Right Performance

Good SDC = Smooth Design Flow = Successful Signoff

Why is SDC Required?

Without Constraints, Tools Don't Know Your Goals!

Constraints Guide the Tools to the Right Design!

1 Why does synthesis require SDC?

- Synthesis (e.g., Genus) needs timing targets to optimize the RTL into a gate-level netlist that meets performance, area and power goals.
- SDC provides clocks, I/O delays, timing exceptions and design rule constraints.
- Without SDC, the tool cannot make timing-driven optimization decisions and may optimize only for area.

RTL (.v) → **Synthesis (Genus)** → Netlist (.v)

SDC = tells the tool “how fast” and “under what conditions” to build the design.

2 Why can't synthesis know the required frequency without SDC?

- The RTL only describes functionality, not performance requirements.
- There is no information about clock frequency, clock period, or timing budget in RTL.
- The tool does not know whether the design should run at 50 MHz, 500 MHz or 1 GHz.

RTL (Functionality only)

```
always @(posedge clk) begin
  q <= d;
end
```

SDC (Performance info)

```
create_clock -period 2.0 [get_ports clk]
```

RTL tells what to build, not how fast to build.

SDC tells the tool the required frequency (e.g., 500 MHz for 2 ns period).

3 What happens if no clock is defined?

- All sequential paths are unconstrained.
- The tool does not know the timing budget.
- Synthesis may optimize only for area (not timing).
- STA cannot calculate setup/hold slack.
- You may get a functionally correct netlist but with unknown (and usually poor) timing performance.

design.sdc No clock defined

Synthesis (Genus)

- No timing targets
- All paths unconstrained
- Optimizes for area
- May run at 500 MHz or 1 GHz
- Timing report not generated

4 What happens if I/O delays are missing?

- Synthesis assumes inputs arrive at time 0 and outputs are required at the clock edge.
- This gives an incorrect (and optimistic) timing budget for internal logic.
- In reality, external chips/PCB have delays, so the internal budget is smaller.
- Result: The design may fail timing in silicon even if synthesis/STA reports look clean.

External (0.3 ns) → PCB Delay (0.5 ns) → External (0.5 ns)

5 What happens if timing exceptions are missing?

- False paths, multicycle paths, and other exceptions inform the tool about paths that are not real timing paths or have relaxed timing requirements.
- Without exceptions, the tool will try to meet timing on all paths, including non-critical or impossible paths (e.g., async resets, CDC paths, scan paths).
- This wastes optimization effort, increases area and power, and may still not meet timing due to unrealistic paths.

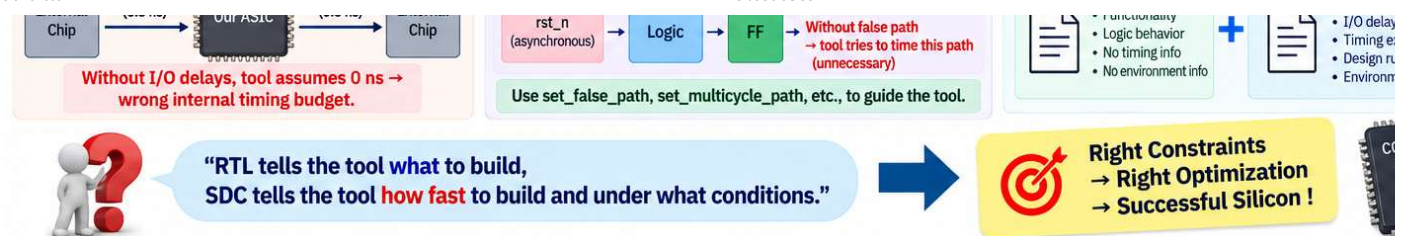
Example: Asynchronous Reset Path

6 Why is RTL alone not sufficient for timing optimization?

- RTL provides only the functional behavior (what the design does).
- It does not specify clocks, timing requirements, I/O exceptions or design rule constraints.
- Multiple correct implementations can exist with the same functional behavior but different performance, area and power.
- SDC provides the additional information needed to guide the implementation to meet timing and design goals.

RTL: What the design does

SDC: How the design should be implemented



What Does SDC Actually Do ?

Three Main Jobs of SDC

Same Design Different Results With vs Without SDC

1 Define the Timing Problem

- Tells the tool how fast the design should run (i.e., clock frequency and timing targets).
- Defines clocks, clock periods, and generated clocks.
- Provides the timing budget for optimization.

Example: Clock Definition

```
create_clock -name clk -period 1.0 [get_ports clk]
```

Without this, the tool does not know how fast to build the design.

2 Define the Design Environment

- Describes the external timing environment (I/O delays, virtual clocks, interface timing).
- Tells the tool when inputs arrive and when outputs are required, based on external chips/PCB.
- Helps calculate the correct internal timing budget.

Example: Input and Output Delays

```
set_input_delay -max 0.3 [get_ports in1]
set_output_delay -max 0.5 [get_ports out1]
```

Without I/O delays, the tool assumes inputs arrive how fast 0 and outputs are required at the clock edge.

3 Define Paths that Should Not be Optimized

- Specifies timing exceptions such as false paths, multicycle paths, and asynchronous clocks.
- Prevents the tool from wasting effort on non-optimal or impossible paths.
- Helps reduce area and power by avoiding unnecessary optimizations.

Example: False Path and Multicycle Path

```
set_false_path -from [get_ports rst_n]
set_multicycle_path 2 -from [get_clocks clk] -to [get_clocks clk_fa]
```

Without exceptions, the tool will try to meet timing on all paths, including unrealistic paths (e.g., reset, asynchronous).

4 How Does SDC Affect Synthesis Optimization?

- Provides timing targets, I/O environment and constraints to guide optimization.
- Helps the synthesis tool (e.g., Genus) make timing-driven decisions such as:
 - Gate sizing
 - Buffer insertion
 - Logic restructuring
 - Area vs. timing trade-offs
- Without SDC, synthesis optimizes only for area (no timing goals).

With SDC → Timing-optimized netlist
Without SDC → Area-optimized netlist (may fail timing)

5 How Does SDC Affect STA?

- Used by STA tool (e.g., Tempus) to verify that the design meets all timing requirements.
- Defines clocks, I/O delays, exceptions, and design rule constraints for a STA tool.
- Allows STA to calculate accurate setup/hold slack and report real timing.
- Without SDC, STA cannot calculate meaningful timing (no clocks, no design rule exceptions).

Right Constraints → Right Optimization → Right Performance

SDC = Tell the Tools What to Achieve, What to Verify, and What to Optimize

Who Creates the SDC File ?

A Collaborative Effort Across Multiple Teams

Different Teams Different Constraints One SDC - A Successful Chip!

1 Who creates the initial SDC?

- The initial SDC is created by your team (usually RTL/Design team) based on the specifications.
- It is refined and updated by various teams (constraints, DFT, low-power, PD) as the design progresses through synthesis, PnR and signoff.

Initial SDC = Starting point (refined by all teams)

2 What does the RTL/design team contribute?

- Clock definitions (primary clocks).
- Clock frequencies and timing targets.
- Generated clocks (from dividers, PLLs, etc.).
- Multicycle path identification (e.g., for slow paths).
- False path identification (e.g., reset paths).

Example:

```
create_clock -name clk \
-period 1.0 [get_ports clk]

create_generated_clock -name clk_div \
-source [get_ports clk] \
-divide_by 2 [get_pins u_div/q]
```

3 What does the constraints team contribute?

- I/O delay values (input and output).
- Interface timing specifications.
- Board-level timing analysis (e.g., PCB delays).
- Virtual clock definitions (for external interfaces).
- Environment assumptions (at chip boundaries).

Example:

```
set_input_delay -max 0.3 \
[get_ports adc_data]

set_output_delay -max 0.5 \
[get_ports data_out]
```

4 What does the DFT team contribute?

- Scan mode constraints.
- set_case_analysis for scan_enable and test mode signals.
- False paths for scan chains (in functional mode).
- Test clock definitions (if separate).

Example:

```
set_case_analysis 0 [get_ports scan_en]

set_false_path -from [get_ports scan_in] \
-to [get_ports scan_out]
```

5 What does the low-power team contribute?

- Power mode constraints (multi-mode SDC).
- set_case_analysis for power control signals (e.g., sleep, isolation).
- Constraints for isolation cells, level shifters, and retention logic.
- Timing for power-up / power-down sequences (if required).

Example:

```
set_case_analysis 0 [get_ports sleep_en]

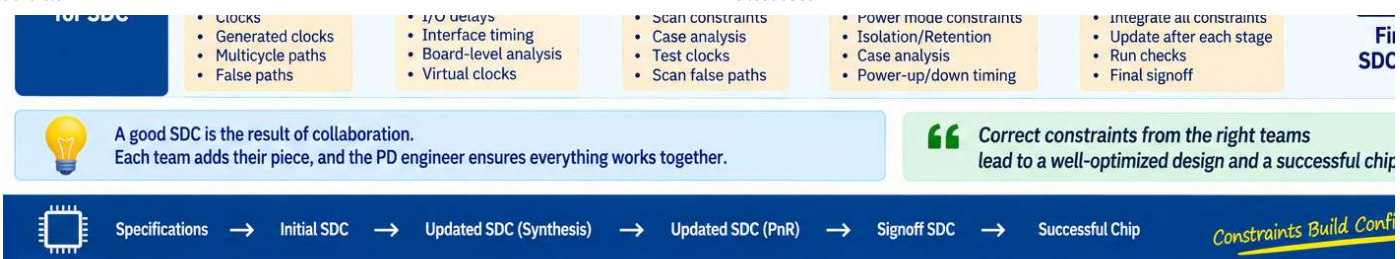
set_false_path -from [get_ports iso_en] \
-to [get_cells ISO_CELL*]
```

6 What does the PD engineer contribute?

- Integrate all constraints from different teams.
- Ensure the SDC is consistent and complete.
- Update SDC after PnR (tool constraints).
- Run checks (e.g., for missing or conflicting constraints).
- Work with STA for signoff SDC.

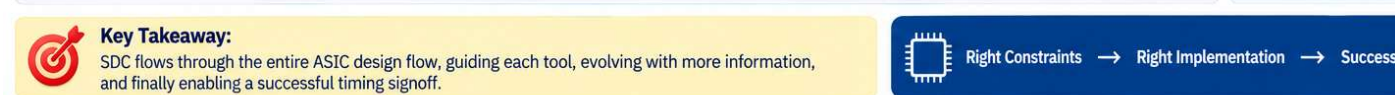
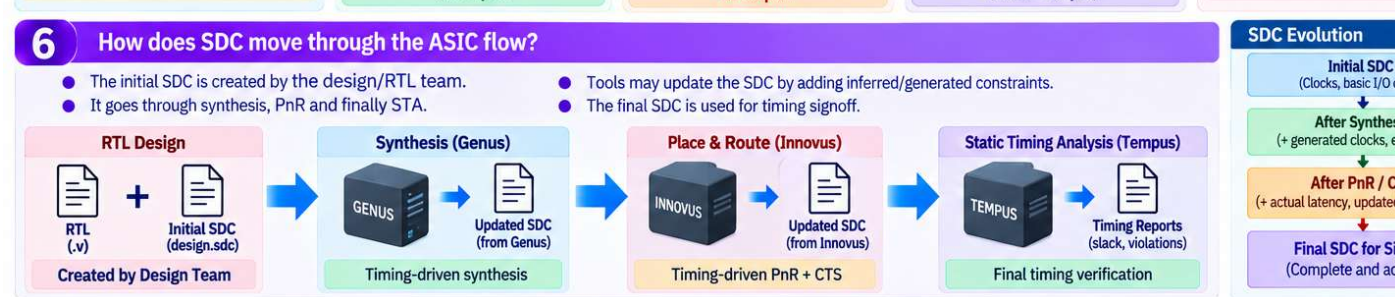
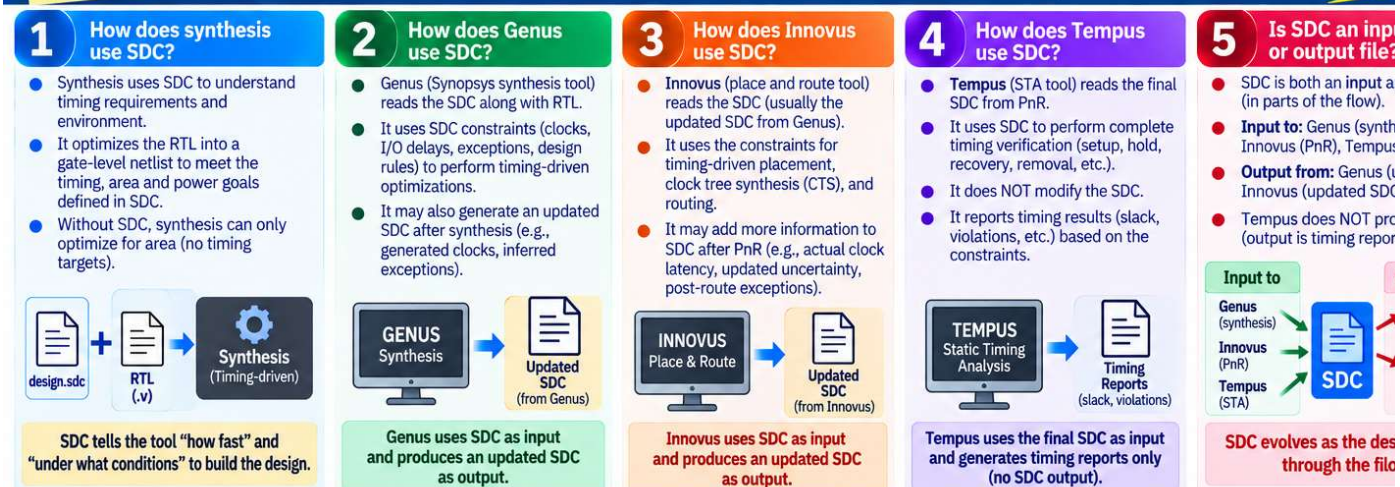
PD Eng (Integration)





SDC in the ASIC Flow

Same Constraints. Different Uses. A Successful Chip.

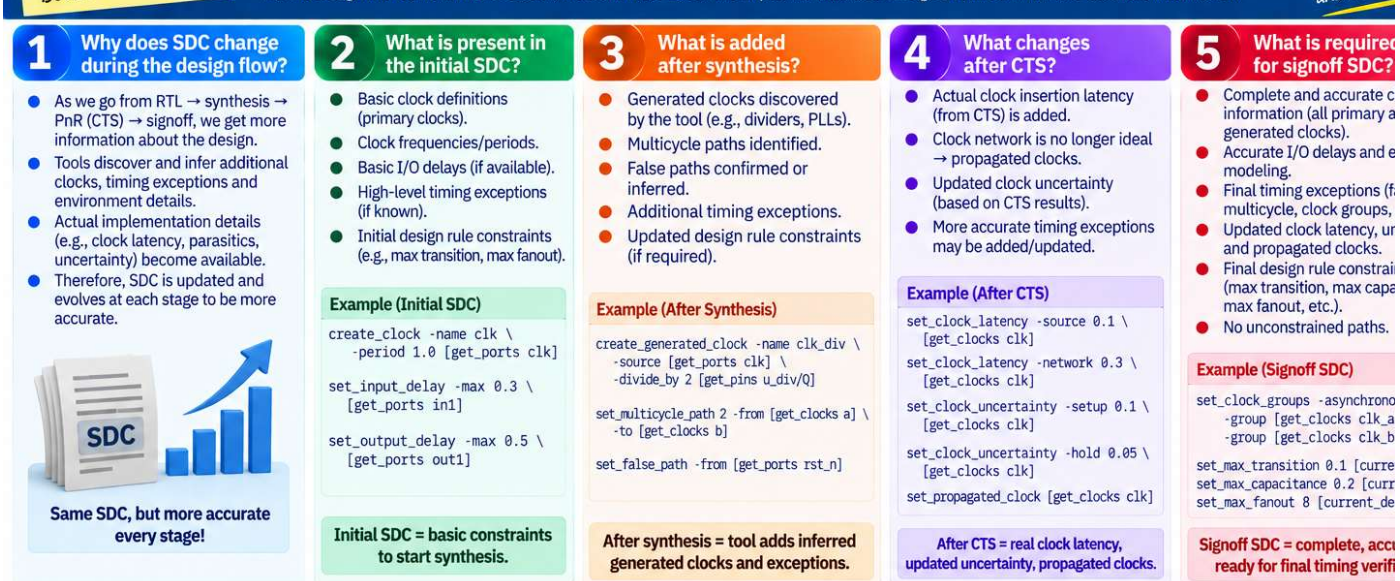


Why Does SDC Evolve?

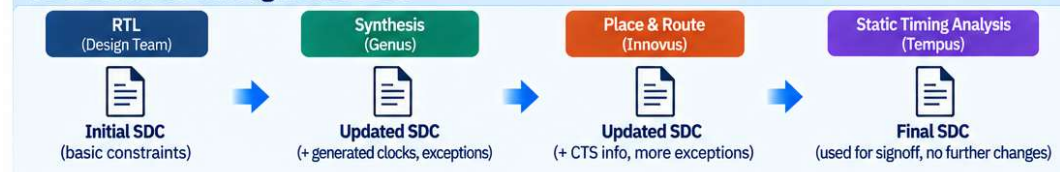
Same Design. More Information. Better Constraints!

As the design moves from RTL to Silicon, we learn more about the implementation, so SDC gets refined with more accurate information.

Evolving SDC Leads to Better Timing and a Successful Chip



SDC Evolution Through ASIC Flow



Key Takeaway

"SDC evolves because we learn more about the design as it is implemented. A correct and complete SDC is essential for accurate timing analysis and a successful silicon."



Initial Constraints → Refined Constraints → Implementation Details → Accurate Timing → Successful Silicon !

Better Information
Better Results

Same Design
Same SDC (Evolving)
Better Implementation !

Initial, Post-Synthesis and Post-CTS SDC

How SDC Evolves Through the Design Flow

More Information
at Each Stage
= More Accurate

1 Pre-Synthesis SDC (Initial SDC)

What is pre-synthesis SDC?

- Initial SDC created by design/RTL team based on the specification.
- Contains basic constraints to guide synthesis.
- Includes primary clocks, basic I/O delays and high-level exceptions (if known).



Example contents:

- create_clock
- set_input_delay
- set_output_delay
- Basic exceptions
- Basic design rule constraints

Used as input to synthesis (Genus)

2 Post-Synthesis SDC

What is post-synthesis SDC?

- Updated SDC generated by synthesis tool (e.g., Genus).
- Includes all initial constraints + tool inferred information.
- Adds generated clocks, inferred exceptions and updated design rule constraints.



Example additions:

- Generated clocks (e.g., from dividers, PLLs)
- Multicycle paths identified
- False paths (inferred)
- Updated design rules

Used as input to PnR (Innovus)

3 Post-CTS SDC

What is post-CTS SDC?

- Updated after clock tree synthesis (CTS) in PnR (Innovus).
- Includes actual clock insertion latency and propagated clocks.
- Updates clock uncertainty based on CTS results.
- May add more accurate exceptions and environment details.



Example additions:

- Actual clock latency (from CTS)
- Updated uncertainty
- set_propagated_clock
- Refined exceptions

Used for final STA (Tempus)

4 Signoff SDC

What is signoff SDC?

- Final and complete SDC used for timing signoff.
- Contains accurate and final constraints based on post-layout information.
- Includes all clocks, latencies, and I/O delays, exceptions and design constraints.
- No unconstrained paths should remain.



Example contents:

- Complete clock definition
- Final I/O delays
- Final exceptions
- Final design rule constraints (max transition, capacitance)
- Accurate uncertainty

Used for signoff timing analysis (Tempus)

ASIC Design Flow and SDC Evolution



5 Who owns SDC at different stages?

Stage	Owner / Responsible Team
Pre-Synthesis (Initial SDC)	RTL / Design Team (sometimes Constraints team)
Post-Synthesis SDC	Synthesis Team (Genus)
Post-CTS SDC	Physical Design Team (Innovus)
Signoff SDC	PD Team + STA Team (Tempus) ... final signoff

6 What constraints change at every stage?

Constraint Type	Initial SDC	After Synthesis	After CTS	Signoff SDC
Primary clocks	✓	✓	✓	✓
Generated clocks	-	✓	✓	✓
Clock latency	(estimated)	(estimated)	(actual)	final
Clock uncertainty	(estimated)	(estimated)	(updated)	final
I/O delays	✓	✓	✓	✓
Timing exceptions	(basic)	(added/refined)	(refined)	final
Design rule constraints	(basic)	(updated)	(updated)	final



Key Takeaways

- SDC evolves as more implementation information becomes available.
- Each stage refines and improves the SDC.
- Final signoff SDC must be complete and accurate.
- All teams contribute to SDC at different stages.
- A correct SDC is essential for a successful design.

Better Constraints → Better Timing
A Successful Product !

No Clock
No Timing
No Silicon !

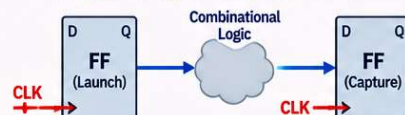
Why is Clock Definition Critical ?

Clock is the heartbeat of timing analysis

Define Clocks...
Meet Timing...
Build a Successful Product

1 Why is a clock required for STA?

- STA checks whether data launched from a launch edge reaches the capture edge in time.
- Clock provides the timing reference (edges, period, waveform) for all timing calculations.
- Without a clock, STA cannot determine setup/hold timing, path delays or timing requirements.



Clock provides the time reference for STA to measure and verify paths.

2 What happens if a clock is missing?

- STA cannot calculate setup/hold timing.
- Paths are reported as unconstrained.
- Tool does not know launch/capture edges.
- Design may appear "timing clean" but it is meaningless.



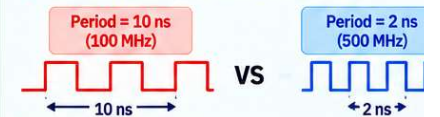
Example STA Report (missing clock)

Warning: No clocks defined in design.
Info : All paths are unconstrained.
Info : No timing analysis performed.

Without a clock, STA has no timing targets, so all paths become unconstrained.

3 What happens if the clock period is wrong?

- If the period is too large (slow clock) → design may meet timing easily, but actual silicon may fail (real time is faster).
- If the period is too small (fast clock) → tool may over-optimize, leading to larger area, higher power and longer runtime.
- Incorrect period gives wrong timing budget.



Correct clock period is essential to meet real performance targets without over- or under-optimization.

4 Why does every timing calculation depend on the clock?

- Setup time: data must arrive before the next clock edge.
- Hold time: data must stay stable after the clock edge.
- Clock period defines the available timing budget.
- All timing paths (register-to-register, I/O, generated clocks) are measured with respect to clock edges.

5 Why is a wrong clock dangerous?

- Chip may fail to meet required performance.
- Can lead to silicon failure even if STA shows no violations.
- May hide real timing issues (if period is too relaxed).
- May cause unnecessary area/power/timing effort (if period is too tight).
- Also affects derived/generated clocks and all related domains.

6 Why must all clock domains be constrained?

- Modern SoCs have multiple clock domains (e.g., different frequencies, PLLs, dividers, async domains).
- Each domain has its own timing requirements.
- Generated clocks (dividers, PLL outputs, gated clocks) must be defined.
- Without constraining all domains, many paths remain unconstrained.

CLK

Data

Valid Data

Setup Time

Hold Time

Clock edges define when data is launched and captured, so every timing calculation is relative to the clock.

Wrong Clock = Risky Silicon

A wrong clock can lead to wrong conclusions, wasted effort or even a failed chip.

- Timing failure
- Higher cost (area/power)
- Missed performance target
- Potential silicon re-spin

Proper clock constraints help in correct timing analysis CDC verification and overall signoff.

PLL

CPU Domain (1 GHz)

Bus Domain (500 MHz)

Peripheral Domain (100 MHz)

All clock domains be consistent

Right Clocks → Right Timing Analysis → Right Optimization → Successful Silicon

Clock Constraints Build Chips

Understanding **create_clock**

Define the Heartbeat of Your Design

A Correct Clock Constraint Today
A Successful Chip Tomorrow!

✓ Define Constraints
✓ Optimize Design
✓ Meet Timing

1 What is create_clock?

- create_clock** is an SDC command used to define a clock in the design.
- It tells the tools (synthesis, PnR, STA) about the clock name, period, waveform and the object on which the clock is created.
- It is the most important clock constraint in SDC.

Syntax (basic):

```
create_clock -name <clk_name> \
-period <time> [options] <object>
```

create_clock defines the timing reference for the entire analysis.

2 How do you create a source clock?

- A source clock is typically created on a primary input port where the clock enters the chip from an external source.
- You specify the clock name, period and (optionally) waveform, and the input port as the object.

Example: Create a source clock on input port

```
create_clock -name clk \
-period 10.0 [get_ports clk]
```

Clock created on input port 'clk' from external source.

3 What is -name?

- name** specifies the user-defined name for the clock.
- It is used to reference the clock in other constraints (e.g., set_clock_latency, set_clock_uncertainty, etc.).
- The name can be any valid identifier (e.g., clk, core_clk, pl).

Example:

```
create_clock -name core_clk -period 5.0 [get_ports
```

Clock Name Usage

- Helps to identify the clock
- Used in other SDC commands
- Should be meaningful
- Case sensitive

Example Usage Later:

```
set_clock_latency 0.2 [get_clocks core_clk]
set_clock_uncertainty 0.1 [get_clocks core_clk]
```

4 What is -period?

- period** specifies the clock time period (in nanoseconds).
- It is the time between two consecutive rising edges of the clock.
- Frequency and period are related as:

$$\text{Frequency (MHz)} = \frac{1000}{\text{Period (ns)}}$$

Example:

```
create_clock -name clk -period 10.0 [get_ports clk]
```

If period = 10 ns → Frequency = 100 MHz

5 What is -waveform?

- waveform** defines the shape of the clock, i.e., when the rising and falling edges occur within one period.
- It is specified as two values: <rise_time> <fall_time> (both in ns).
- If not specified, a 50% duty cycle is assumed (waveform {0 period/2}).

Example:

```
create_clock -name clk -period 10.0 -waveform {0 4} \
[get_ports clk]
```

In this example:

- High time = 4 ns
- Low time = 6 ns
- Duty cycle = 40%

6 What object can a clock be created on

- A clock can be created on the following objects:
 - Input port** (most common for source clocks)
 - Output port** (for generated or forwarded clocks)
 - Pin** (e.g., divider output, PLL output)
 - Net** (less common, usually derived internally)
 - Register** (rare, mostly for special cases)

Examples:

```
# On input port
create_clock -name clk -period 10 [get_ports clk]

# On a pin (e.g., PLL output)
create_clock -name pll_clk -period 5 [get_pins u_pll/clk_out]

# On an output port
create_clock -name out_clk -period 8 [get_ports clk_out]
```

Most source clocks created on input

Key Takeaway:
create_clock tells the tool "what the clock is" — its name, period, waveform and where it comes from. A correct clock definition is the foundation for accurate timing analysis.

Clocks Drive Performance

Clock Period and Frequency

Two Sides of the Same Clock – Define Performance, Area and Power!

Right Clock
Right Performance
Right Silicon!

A Clock Today
A Faster Tomorrow

1 What is clock period?

- Clock period is the time taken for one complete cycle of the clock signal.
- It is usually specified in nanoseconds (ns).
- It is the time between two consecutive rising edges (or falling edges) of the clock.

CLK

Clock Period (T)

Example: If the clock completes one cycle in 10 ns, then the clock period T = 10 ns.

2 What is clock frequency?

- Clock frequency is the number of clock cycles per second.
- It is usually specified in Hertz (Hz) or Megahertz (MHz).
- It tells how fast the clock is running.

Example:
If the clock frequency is 100 MHz, it means 100 million cycles every second.

100 MHz
100,000,000 cycles per second

3 What is the relationship between period and frequency?

- Clock period (T) and clock frequency (f) are inversely related.
- If frequency increases, period decreases, and vice versa.

$$T = 1 / f \quad \text{or} \quad f = 1 / T$$

Frequency (f)	Period (T)
50 MHz	20 ns
100 MHz	10 ns
200 MHz	5 ns
500 MHz	2 ns
1 GHz	1 ns

Key Point:
Higher frequency means smaller period and a tighter timing requirement.

Higher Frequency = Faster Design

4 How does clock period define timing budget?

- The clock period is the total time available for data to propagate from a startpoint to an endpoint (e.g., from a register to a register).
- All delays (combinational logic, interconnect, setup time, clock uncertainty, etc.) must fit within this period.

5 What happens if the period is too tight?

- The design has less time to meet timing.
- Tool may need to use larger cells, add pipeline stages, or optimize heavily.
- Can lead to higher area, higher power and more design effort.
- If timing cannot be met, the design will fail (setup violations).

Example: Trying to run at 1 GHz (T = 1 ns) means every path must complete within 1 ns, which is very challenging.

6 What happens if the period is too relaxed?

- The design has more time to meet timing.
- Tool may choose smaller, lower-drive cells.
- Leads to lower performance (slower design).
- May result in larger area (if not optimized) and miss performance targets.

Example: If T = 20 ns (50 MHz), the design is easier to meet timing, but the chip will be slower and may not satisfy performance targets.

Key Takeaway:
Clock period and frequency define the speed of the design. Choosing the right clock period is a trade-off between performance, area, power and design effort.

Tight Period (High Frequency)

$T = 2 \text{ ns}$
(500 MHz)

- Harder to meet timing
- May need bigger cells
- Higher area and power
- Risk of setup violations

Relaxed Period (Low Frequency)

$T = 20 \text{ ns}$
(50 MHz)

- Easier to meet timing
- Can use smaller cells
- Lower power (typical)
- Lower performance

*Choose the Right Clock
for the Right Product*

Clock Waveform and Duty Cycle

Shape Matters in Timing !

1 What does -waveform {rise fall} mean?

- The **-waveform {rise fall}** option in **create_clock** specifies the time (in ns) of the rising edge and falling edge within one clock period.
- rise** = time of the first rising edge (usually 0).
- fall** = time of the next falling edge.
- It defines the shape of the clock (high time and low time) within the clock period.

Example:
`create_clock -name clk -period 10.0 -waveform {0 4} [get_ports clk]`

Rising edge at 0 ns
Falling edge at 4 ns
High time = 4 ns
Low time = 6 ns
Period = 10 ns

2 What is a rising edge?

- A rising edge is the transition of the clock signal from logic 0 (low) to logic 1 (high).
- It is also called a positive edge.
- Many sequential elements (e.g., flip-flops) are triggered on the rising edge.

Rising Edge (0 → 1)

Rising edge = low to high transition (0 → 1)

3 What is a falling edge?

- A falling edge is the transition of the clock signal from logic 1 (high) to logic 0 (low).
- It is also called a negative edge.
- Some sequential elements may be triggered on the falling edge.

Falling Edge (1 → 0)

Falling edge = high to low transition (1 → 0)

4 What is a 50% duty cycle?

- A 50% duty cycle means the clock is high for half of the period and low for the other half.
- In this case, fall time = rise time + (period / 2).
- It is the most common clock waveform.

Example: `create_clock -name clk -period 10.0 -waveform {0 5}`

Duty Cycle = 50%

$$\text{Duty Cycle} = \frac{\text{High time}}{\text{Period}} \times 100 = \frac{5}{10} \times 100 = 50\%$$

5 What is a non-50% duty cycle?

- A non-50% duty cycle means the clock is high and low for unequal durations within the period.
- It is defined using **-waveform {rise fall}** where fall - rise ≠ period/2.
- Non-50% duty cycles are common in real designs (e.g., due to clock gating, PLLs, external sources).

Example: `create_clock -name clk -period 10.0 -waveform {0 3}`

Duty Cycle = 30%

$$\text{Duty Cycle} = \frac{3}{10} \times 100 = 30\%$$

6 Why does clock waveform matter?

- The clock waveform defines the actual timing of rising and falling edges.
- Setup and hold checks depend on the correct edge.
- An incorrect waveform can give wrong timing analysis.
- Non-50% duty cycles affect:
 - Time available for data (high/low time)
 - Generated clocks (e.g., dividers, PLLs)
 - Clock gating and power analysis
 - Multi-cycle and edge-based constraints

✓ Accurate clock waveform definition is essential for correct timing analysis, optimization and verification.

Key Takeaway:

The **-waveform {rise fall}** defines the clock shape within one period. It determines the edge timings, duty cycle and available time for data. A correct clock waveform ensures accurate timing analysis and a successful silicon !

*Right Waveform
Right Timing
Right Chip*

Clock Waveform and Duty Cycle

Shape Matters in Timing !

1 What does -waveform {rise fall} mean?

- The **-waveform {rise fall}** option in **create_clock** specifies the time (in ns) of the rising edge and falling edge within one clock period.
- rise** = time of the first rising edge (usually 0).
- fall** = time of the next falling edge.
- It defines the shape of the clock (high time and low time) within the clock period.

Example:
`create_clock -name clk -period 10.0 -waveform {0 4} [get_ports clk]`

Rising edge at 0 ns
Falling edge at 4 ns
High time = 4 ns
Low time = 6 ns
Period = 10 ns

2 What is a rising edge?

- A rising edge is the transition of the clock signal from logic 0 (low) to logic 1 (high).
- It is also called a positive edge.
- Many sequential elements (e.g., flip-flops) are triggered on the rising edge.

Rising Edge (0 → 1)

Rising edge = low to high transition (0 → 1)

3 What is a falling edge?

- A falling edge is the transition of the clock signal from logic 1 (high) to logic 0 (low).
- It is also called a negative edge.
- Some sequential elements may be triggered on the falling edge.

Falling Edge (1 → 0)

Falling edge = high to low transition (1 → 0)

4 What is a 50% duty cycle?

- A 50% duty cycle means the clock is high for half of the period and low for the other half.
- In this case, fall time = rise time + (period / 2).
- It is the most common clock waveform.

Example: `create_clock -name clk -period 10.0 -waveform {0 5}`

Duty Cycle = 50%

$$\text{Duty Cycle} = \frac{\text{High time}}{\text{Period}} \times 100 = \frac{5}{10} \times 100 = 50\%$$

5 What is a non-50% duty cycle?

- A non-50% duty cycle means the clock is high and low for unequal durations within the period.
- It is defined using **-waveform {rise fall}** where fall - rise ≠ period/2.
- Non-50% duty cycles are common in real designs (e.g., due to clock gating, PLLs, external sources).

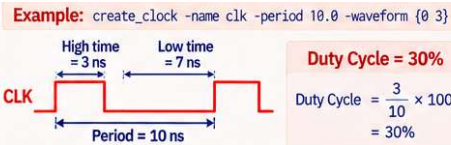
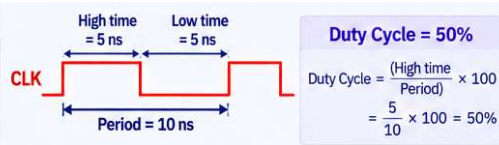
Example: `create_clock -name clk -period 10.0 -waveform {0 3}`

Duty Cycle = 30%

$$\text{Duty Cycle} = \frac{3}{10} \times 100 = 30\%$$

6 Why does clock waveform matter?

- The clock waveform defines the actual timing of rising and falling edges.
- Setup and hold checks depend on the correct edge.
- An incorrect waveform can give wrong timing analysis.
- Non-50% duty cycles affect:
 - Time available for data (high/low time)



- Generated clocks (e.g., dividers, PLLs)
- Clock gating and power analysis
- Multi-cycle and edge-based constraints

✓ Accurate clock waveform definition is essential for correct timing analysis, optimization and



Key Takeaway:

The `-waveform {rise fall}` defines the clock shape within one period. It determines the edge timings, duty cycle and available time for data. A correct clock waveform ensures accurate timing analysis and a successful silicon!

Right Wave
Right Time
Right Ch

One Source
Many Clocks
Same Relationship!

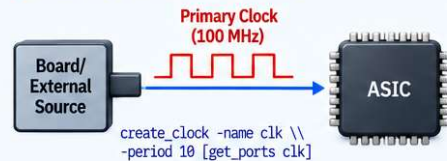
Primary Clock and Generated Clock

Understand the Source, Relationship and Usage in SDC

Define Clocks Correctly
Analyze Timing Accurately
Build a Better Chip

1 What is a primary clock?

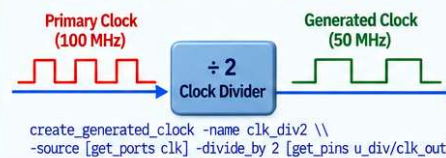
- A primary clock is a clock that is created directly on a primary input port (coming from outside the chip).
- It is defined using `create_clock` in SDC.
- It is the root clock of the design and usually comes from an external source (e.g., oscillator, board clock).
- It has no parent clock inside the chip.



Primary clock is defined at the chip input port.

2 What is a generated clock?

- A generated clock is a clock that is derived from an existing clock inside the chip.
- It is defined using `create_generated_clock` in SDC.
- It can have a different frequency, phase, or waveform compared to its parent clock.
- It is typically created on the output of a clock-generating element (e.g., divider, PLL, mux, gate, etc.).



Generated clock is derived from an existing clock.

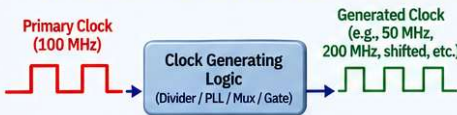
3 What is the difference between primary and generated clocks?

Feature	Primary Clock (<code>create_clock</code>)	Generated Clock (<code>create_generated_clock</code>)
Source	External (chip input port)	Derived from a clock inside the chip
SDC Command	<code>create_clock</code>	<code>create_generated_clock</code>
Where defined	On input port	On internal pin (output of clock-generating element)
Has parent clock?	No	Yes
Frequency	Defined by user	Related to parent (e.g., divide, multiply)
Examples	Board clock, oscillator clock	Divider clock, PLL clock, muxed clock, gated clock
Purpose	Defines the root clock in the design	Defines derived clocks for accurate timing analysis

Primary clock is the root; generated clock is derived.

4 Where does a generated clock come from?

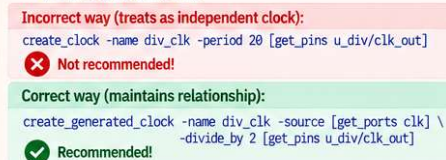
- A generated clock comes from a clock-generating element inside the chip, such as:
 - Clock divider ($\div N$)
 - PLL (multiply/divide, phase shift)
 - Clock mux
 - Clock gate (with enable)
 - Frequency divider in IP blocks
 - Any logic that modifies the clock (e.g., a flip-flop based divider)



Generated clocks originate from internal clock logic.

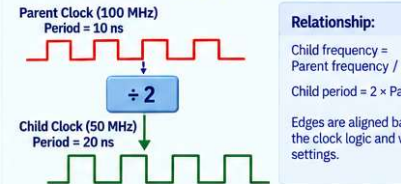
5 Why can't all clocks use `create_clock`?

- `create_clock` should only be used for primary (external) clocks.
- If you use `create_clock` on an internal derived clock, the tool will treat it as an unrelated clock, losing the relationship with the parent clock.
- This can lead to incorrect timing analysis, false violations, or missing checks.
- Generated clocks must use `create_generated_clock` so the tool understands their relationship with the parent clock (frequency, phase, etc.).



6 How are parent and child clocks related?

- A generated clock (child) is always related to a parent clock.
- The child clock inherits its timing information (frequency, phase, waveform) from the parent clock, based on the relationship defined in the SDC.
- The relationship can be defined using options like `-divide_by`, `-multiply_by`, `-phase`, or `-source`.



Key Takeaway:

Use `create_clock` for primary clocks and `create_generated_clock` for derived clocks. Maintain the correct parent-child relationship for accurate timing analysis and successful signoff.

Correct Clock Constraints
Lead to Correct Timing

Derived Clocks
Correct Constraints
Better Silicon!

Understanding `create_generated_clock`

Define Derived Clocks Correctly for Accurate Timing Analysis

Same Clock
Different Constraints
Same Silicon

1 What is `create_generated_clock`?

- `create_generated_clock` is an SDC command used to define a clock that is derived from an existing clock (primary or another generated clock).
- It describes the relationship between the generated clock and its source clock (frequency, phase, waveform).
- It is used for clocks created inside the chip (e.g., by dividers, PLLs, clock muxes, clock gating, etc.).

Syntax (common form):

```
create_generated_clock -name <clk_name> \
-source <source_clk> \
[-divide_by <N>] [-multiply_by <N>] [-phase <deg>] \
[options] <object>
```

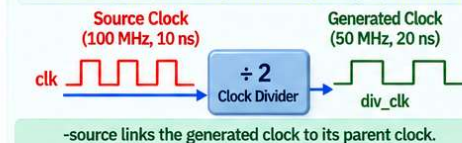
`create_generated_clock` tells the tool how a derived clock is related to its source clock.

2 What is `-source`?

- `-source` specifies the parent clock from which the generated clock is derived.
- It can be a clock name or a port/pin with a clock.
- It tells STA which clock to use as the reference for frequency, phase and waveform.

Example:

```
create_generated_clock -name div_clk \
-source [get_ports clk] -divide_by 2 [get_pins div/Q]
```

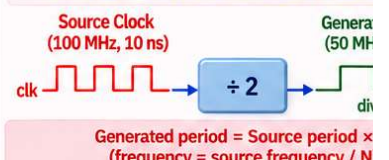


3 What is `-divide_by`?

- `-divide_by` specifies that the generated clock is the source clock frequency divided by N.
- It is commonly used for clock dividers.
- The waveform and phase are also adjusted.

Example (divide by 2):

```
create_generated_clock -name div_clk \
-source [get_ports clk] -divide_by 2 [get_pins div/Q]
```



4 What is `-multiply_by`?

- `-multiply_by` specifies that the generated clock frequency is the source clock frequency multiplied by N.
- It is used for clock multipliers (e.g., PLLs, frequency multipliers, DDR clocks).
- The waveform is scaled according to the new period.

Example (multiply by 4):

5 Where is the generated clock created?

- The generated clock is created on the output of the clock-generating element inside the chip.
- The `<object>` specifies where the generated clock is defined (e.g., a pin, port, or net).
- Common examples: output pin of a divider, PLL output, mux output, gated clock output, etc.

Example (on a pin):

6 How does STA calculate the generated clock period?

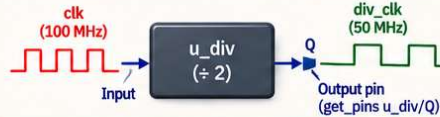
- STA uses the relationship defined in the SDC command (e.g., `-divide_by`, `-multiply_by`, `-phase`, `-source`) to the generated clock period.
- It takes the source clock period and applies the scaling transformation.
- The generated clock also inherits waveform and phase (unless explicitly overridden).

```
create_generated_clock -name mul_clk \
-source [get_ports clk] -multiply_by 4 [get_pins pll_out]
```



Generated period = Source period / N
(frequency = source frequency $\times$ N)

```
create_generated_clock -name div_clk \
-source [get_ports clk] -divide_by 2 [get_pins u_div/Q]
```



The generated clock is created on the specified object (usually the downstream output pin or net).

Calculation Examples:

Case	Source Period	SDC Option	Ge
Divide by 2	10 ns (100 MHz)	-divide_by 2	20
Divide by 4	10 ns (100 MHz)	-divide_by 4	40
Multiply by 4	20 ns (50 MHz)	-multiply_by 4	5
Multiply by 2	10 ns (100 MHz)	-multiply_by 2	5

STA automatically calculates the generated clock period based on the SDC definition and the source clock.



Key Takeaway:

Use `create_generated_clock` with the correct `-source`, `-divide_by` / `-multiply_by` and `<object>` to define derived clocks. This allows STA to accurately calculate the generated clock period, phase and waveform, ensuring correct timing analysis and signoff.

Accurate
Lead to Reli

Different Circuits
Different Clocks
Same Methodology !

Generated Clock Examples

From Dividers to PLLs – Constrain Every Clock Correctly !

Accurate
Better T
Success

1 How is a divide-by-2 clock created?

- A divide-by-2 clock is created using `create_generated_clock` with `-divide_by 2`.
- It is derived from a parent clock and has half the frequency (double the period).

SDC Example:

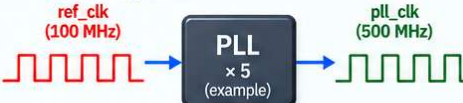
```
create_generated_clock -name div2_clk \
-source [get_ports clk] -divide_by 2 \
[get_pins u_div/Q]
```



Frequency is divided by 2, period is doubled.

2 How is a PLL-generated clock defined?

- A PLL generates a clock with a multiplied/divided frequency and possibly a phase shift.
- Use `create_generated_clock` with `-multiply_by`, `-divide_by` and optionally `-phase`.



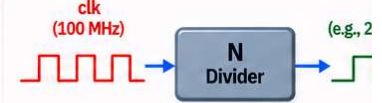
SDC Example:

```
create_generated_clock -name pll_clk \
-source [get_ports ref_clk] -multiply_by 5 \
[get_pins u_pll/clk_out]
```

Generated clock frequency = Parent frequency $\times$ 5
(Period = 2 ns if parent is 100 MHz).

3 How is a clock from a divider defined?

- A clock from a divider (e.g., counter or clock divider) is defined using `create_generated_clock` with `-source` and `-divide_by`.
- The generated clock is created on the output pin of the divider.



SDC Example (divide by 4):

```
create_generated_clock -name div4_clk \
-source [get_ports clk] -divide_by 4 \
[get_pins u_divider/clk_out]
```

Generated clock period = Parent period $\times$ 4

4 How is a clock from a MUX handled?

- A MUX can select between multiple clocks.
- Use `create_generated_clock` with `-source` and specify the MUX output as the generated clock.
- If multiple clocks can pass through, use `-master_clock` (or `set_clock_groups` if mutually exclusive).



SDC Example:

```
create_generated_clock -name mux_clk \
-source [get_ports clk1] \
[get_pins u_mux/clk_out]
```

The MUX output clock inherits the frequency/phase of the selected input.

5 What is a phase-shifted generated clock?

- A phase-shifted clock has the same frequency as the parent but with a defined phase shift.
- Use `create_generated_clock` with `-phase` to specify the phase shift (in degrees).



SDC Example (90° phase shift):

```
create_generated_clock -name clk_90 \
-source [get_ports clk] -phase 90 \
[get_pins u_phase/clk_out]
```

Frequency remains the same, but the waveform is shifted in time.

6 What happens if a generated clock is not defined?

- The tool will not know about the derived clock.
- Paths driven by the missing clock may not be analyzed with wrong assumptions.
- This can lead to:
 - Missed timing violations
 - Optimistic (incorrect) timing results
 - Functional failures in silicon
 - Signoff mismatches

Example Scenario:

If a divider output clock is not defined, STA may treat a combinational path or use wrong default assumptions, leading to critical violations to be missed.



Key Takeaways:

- Use `create_generated_clock` for all derived clocks (dividers, PLLs, MUXs, phase shifts, etc.).
- Specify correct source, divide/multiply/phase and the output object.
- Never miss a generated clock – accurate constraints ensure correct timing analysis and a successful tapeout !

Not Every Clock
Comes from Silicon !

Virtual Clock

A Logical Clock for I/O Timing – No Physical Source, But Very Important !

Constrain t
Not Just t

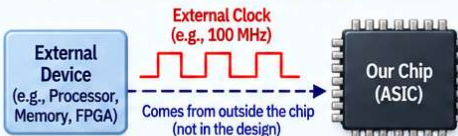
1 What is a virtual clock?

- A virtual clock is a user-defined clock that does not exist physically inside the chip.
- It is used only for timing analysis, typically for I/O paths (ports), not for internal logic.
- It provides a reference for input and output delay constraints (e.g., `set_input_delay`, `set_output_delay`).
- It helps STA analyze timing between the external world and the chip.

A virtual clock is a timing reference, not a real clock in the design.

2 Why does a virtual clock have no physical source?

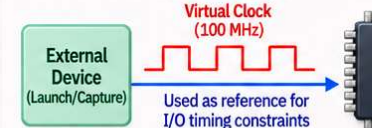
- A virtual clock represents an external clock (from another chip, board, or system) which is not part of the current design.
- Since the source is outside the chip, there is no physical pin/net inside the design to attach the clock.
- Therefore, it is created as a "logical" clock only for constraints.



Virtual clock has no physical source inside the chip because it originates outside the chip.

3 Why is a virtual clock required for I/O timing?

- I/O timing checks (`set_input_delay`, `set_output_delay`) need a clock reference to measure arrival/required times.
- The external device provides a clock, but it is not part of the design, so we use a virtual clock to model it.
- It allows STA to correctly analyze the timing between the external world and the chip I/O ports.



Virtual clock enables accurate I/O timing analysis with external devices.

4 How do you create a virtual clock?

- Use the `create_clock` command without specifying any port or net (or use a dummy name).
- You only define the clock name, period (and optionally waveform).

5 What is the difference between virtual and physical clocks?

Feature	Virtual Clock	Physical Clock
Exists in design?	No	Yes
Source	External (outside chip)	On-chip or chip input port

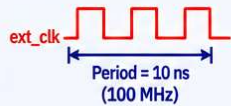
6 When should a PD engineer use a virtual clock?

- When constraining input and output delays.
- When the clock source is outside the chip (e.g., memory, or another ASIC/FPGA).
- When no physical clock port exists in the design.

- It is not attached to any design object.

SDC Example:

```
create_clock -name ext_clk -period 10.0
(Optionally with waveform)
create_clock -name ext_clk -period 10.0 -waveform {0 5}
```



No port or net is specified
→ It is a virtual clock.

Source	External (outside chip)	On-chip (on-chip input port)
Created on	No port/net (logical only)	Port, net or pin
Used for	I/O timing (input/output delays)	Internal timing and I/O timing
Command	create_clock (no object)	create_clock (on port/net)
Drives logic?	No	Yes
Examples	External memory clock, CPU clock, test equipment clock	System clock, PLL clock, divided clock, gated clock

Virtual clock = external reference for I/O.
Physical clock = real clock used inside the chip.

- When no physical clock port exists in the design, an external clock is used.
- During top-level chip integration to model external clock.
- Also used for test mode constraints (e.g., ATF when the source is not in the design).

Common Use Cases:

- Input timing (set_input_delay)
- Output timing (set_output_delay)
- External memory interfaces (DDR, LPDDR, etc)
- Chip-to-chip interfaces
- Test mode and DFT constraints

**Key Takeaway:**

A virtual clock is a logical clock used for I/O timing. It has no physical source inside the chip but is essential to accurately constrain and analyze external interfaces.

Good Constraints
Make a Better Design

In Theory: Perfect
In Silicon: Imperfect
Know the Difference!

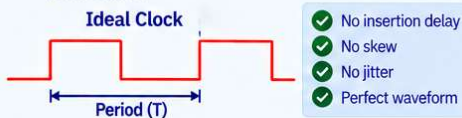
Ideal Clock vs Real Clock

From a Perfect Waveform to a Real Silicon Clock – Understand the Differences!

A Real Clock has
Delays, Skew and
Jitter But Still Makes

1 What is an ideal clock?

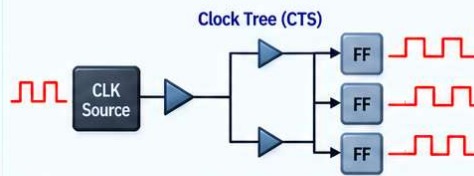
- An ideal clock is a perfect, hypothetical clock used during pre-CTS analysis.
- It has no insertion delay, no skew, no jitter, and an ideal waveform (perfect rise/fall, fixed period).
- It is assumed to arrive at all sequential elements simultaneously.
- Used for early timing analysis (pre-CTS) to estimate performance.



Used in pre-CTS (ideal clock mode)

2 What is a propagated clock?

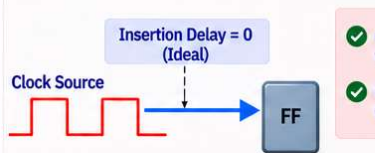
- A propagated clock is a real clock that propagates through the actual clock network (buffers, inverters, CTS tree) in the design.
- It includes insertion delays, skew and jitter.
- Used for post-CTS analysis to get accurate timing.



Used in post-CTS (propagated clock mode)

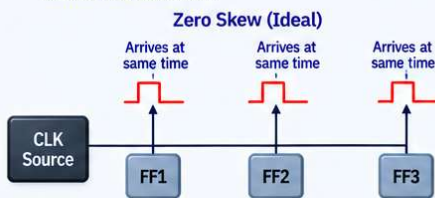
3 What is zero insertion delay?

- Zero insertion delay means the clock is assumed to arrive at every flip-flop with 0 delay from the clock source.
- Used in ideal clock mode (pre-CTS).
- Simplifies early timing analysis by ignoring delays.



4 What is zero skew?

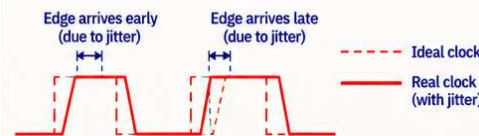
- Zero skew means the clock arrives at all flip-flops at the same time (no time difference).
- Assumed in ideal clock mode (pre-CTS).
- In real designs, some skew always exists due to clock network variations.



In ideal clock, all FFs see the clock edge simultaneously.

5 What is clock jitter?

- Clock jitter is the short-term variation in the clock edge arrival time from its ideal position.
- Caused by PLL noise, power supply noise, crosstalk, process variations, etc.
- Jitter reduces the effective timing margin and is considered during signoff (setup/hold analysis).



Jitter causes uncertainty in the clock edge, which reduces timing margin.

6 Why is a pre-CTS clock different from a post-CTS clock?

- Pre-CTS clock is an ideal clock with zero insertion delay and no jitter.
- Post-CTS clock is a propagated clock with real delays, skew and jitter.
- After CTS, the actual clock network (buffers, inverters) introduces delays and variations, so the clock is no longer ideal.
- Post-CTS analysis gives accurate timing results, while pre-CTS is only an optimistic estimate.

Aspect	Pre-CTS (Ideal Clock)	Post-CTS (Propagated Clock)
Insertion Delay	0	Non-zero
Skew	0	Non-zero
Jitter	0	Non-zero
Clock Network	Not considered	Included
Accuracy	Optimistic estimate	Realistic
Used for	Early analysis (synthesis)	Post-CTS analysis

**Key Takeaway:**

Ideal clocks are used for early, simplified analysis, while propagated clocks model the real clock behavior with delays, skew and jitter. Always analyze with propagated clocks after CTS for accurate timing results!

Understand
Build Reliable Design

Same Clock,
Different Realities,
Add Uncertainty!

set_clock_uncertainty

Account for Clock Variations – Get Realistic Timing Results!

Be Pessimistic
Constraints,
Be Successful

1 What is set_clock_uncertainty?

- set_clock_uncertainty is an SDC command used to model the uncertainty in the clock.
- It accounts for variations such as clock jitter, clock skew, and modeling inaccuracies.
- It tells STA to be more pessimistic by reducing the available timing budget.
- It can be specified separately for setup and hold checks.
- Helps get realistic timing results, especially before CTS when actual clock network is unknown.

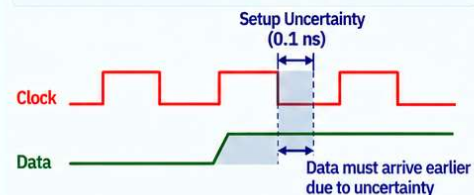
set_clock_uncertainty adds a timing margin to account for clock variations (jitter, skew, etc.).

2 How do you define setup uncertainty?

- Setup uncertainty reduces the available time for data to arrive before the clock edge.
- Use the -setup option to specify setup uncertainty.

SDC Example (Setup Uncertainty):

```
set_clock_uncertainty -setup 0.1 [get_clocks clk]
```

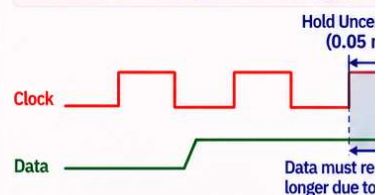


3 How do you define hold uncertainty?

- Hold uncertainty reduces the minimum time the clock edge for data to remain stable.
- Use the -hold option to specify hold uncertainty.

SDC Example (Hold Uncertainty):

```
set_clock_uncertainty -hold 0.05 [get_clocks clk]
```



4 Why is uncertainty used before CTS?

5 How is uncertainty updated after CTS?

6 How does uncertainty reduce the timing budget?

- Before CTS, the actual clock network (skew, insertion delay) is not known.
- Uncertainty models the expected variations (jitter, skew, etc.) in a simple way.
- This prevents overly optimistic timing results.
- It helps find real timing issues early in the flow.

Typical pre-CTS uncertainty values:

- Setup: 0.05 – 0.20 ns (technology dependent)
- Hold: 0.02 – 0.10 ns

Use uncertainty before CTS to be safe and realistic.

- After CTS, the actual clock network is known, including real skew and insertion delays.
- Uncertainty can be updated to smaller values (since some variation is now explicitly modeled in the clock tree).
- Often, tool-recommended or foundry-recommended values are used.
- Separate uncertainties may be used for different clocks or clock domains.

Stage	Setup Uncertainty	Hold Uncertainty	Reason
Pre-CTS	0.05 – 0.20 ns	0.02 – 0.10 ns	Model unknown skew/jitter
Post-CTS	0.01 – 0.05 ns	0.01 – 0.03 ns	Real clock network is known

After CTS, reduce uncertainty (but do not set to zero).

Timing Budget:

- Uncertainty is subtracted from the available timing and reduces the required time for hold.
- This makes the timing checks more difficult to meet.
- It accounts for the worst-case clock variations, design robustness.



Uncertainty reduces the effective timing budget making timing closure more challenging but more realistic.



Key Takeaway:

Use **set_clock_uncertainty** to model clock variations, apply suitable values before and after CTS, and remember that uncertainty reduces your timing budget but gives realistic and reliable results.

Good Constraints Lead to Reliable Results

Same Clock, Different Delays, Different Impact!

Clock Latency

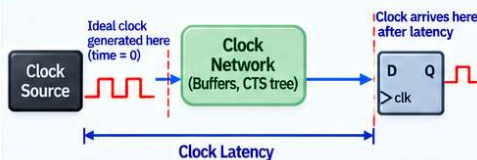
From Clock Source to Flip-Flop – Understand the Delays on the Way!

Account for Get Correct Build Better

1 What is clock latency?

- Clock latency is the time taken for the clock to reach a register (flip-flop) from the clock source.
- It represents the delay between the ideal clock reference point and the actual clock arrival at a point in the design.
- It includes both source latency and network latency.

Clock Latency = Source Latency + Network Latency

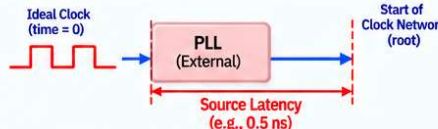


2 What is source latency?

- Source latency is the delay from the ideal clock source (e.g., PLL output, port, or virtual clock) to the start of the clock network (root of the clock tree).
- It models delays outside the design, such as PLL, package, board, or input port delays.
- In many cases, it is user-defined using **set_clock_latency -source**.

Example (Source Latency):

```
set_clock_latency -source 0.5 [get_clocks clk]
```

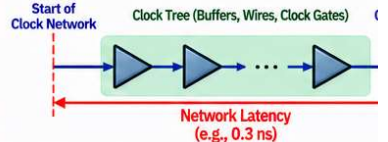


3 What is network latency?

- Network latency is the delay through the on-chip distribution network (clock tree).
- It includes delays from buffers, inverters, clock wires inserted during CTS.
- It is typically calculated by the tool after CTS (set explicitly).

Example (Network Latency):

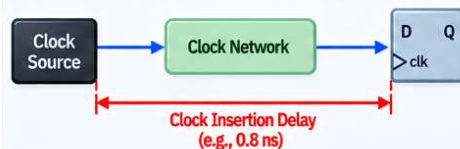
```
set_clock_latency -network 0.3 [get_clocks clk]
```



4 What is clock insertion delay?

- Clock insertion delay is the total delay from the clock source to a specific flip-flop.
- It is the sum of source latency and network latency.
- It can be different for different flip-flops due to clock tree skew and variations.

Clock Insertion Delay = Source Latency + Network Latency



5 What is set_clock_latency?

- set_clock_latency** is an SDC command used to specify source and/or network latency for a clock.
- It can be used before CTS (to model expected delays) or after CTS (to set or override actual values).
- Options:
 - source**: specify source latency
 - network**: specify network latency

Example:

```
set_clock_latency -source 0.5 [get_clocks clk]
set_clock_latency -network 0.3 [get_clocks clk]
# or total latency
set_clock_latency 0.8 [get_clocks clk]
```

Helps STA model realistic clock arrival times and get accurate timing results.

6 How does latency affect setup and hold?

- Clock latency shifts the clock arrival time at the registers which affects the data required and arrival times.
- Source latency affects both setup and hold equally.
- Network latency can be different for launch and capture registers (due to skew), which impacts setup and hold.
- Higher latency reduces the available time for setup (increase) hold time, while lower latency has the opposite effect.

Effect on Setup Timing

- More latency → less time for data to arrive → harder setup.
- Setup slack reduces.
- Longer clock paths can cause setup violations.

Effect on Hold Timing

- More latency (especially capture) → clock arrives later → more hold time.
- Can help avoid hold violations (but depends on skew).
- Less latency can make hold more critical.



Key Takeaway:

Clock latency models the real delays from the clock source to the registers. Use **set_clock_latency** to specify source and network latency, and always consider its impact on both setup and hold timing for accurate STA results.

Model Real Clock Timing

Same Design, Different Analysis, Get Real Timing!

set_propagated_clock

From Ideal Assumptions to Real Delays – Use the Actual Clock Network!

Post-CTS is Use Propagated for Signoff!

1 What is set_propagated_clock?

- set_propagated_clock** is an SDC command used to tell the STA tool to use the actual clock network delays (from CTS) instead of assuming an ideal clock (with zero delay).
- It changes the clock from an ideal clock to a propagated clock in the analysis.

SDC Example:

```
set_propagated_clock [get_clocks clk]
```

This command is usually applied after CTS, when the clock tree is built.

2 Why is it used after CTS?

- After CTS, the actual clock network (buffers, wires, insertion delays, skew) is available in the design.
- Using **set_propagated_clock** ensures STA uses these real delays for more accurate timing analysis.
- It reflects the real clock arrival time at each register.



Use **set_propagated_clock** after CTS to analyze the real clock behavior.

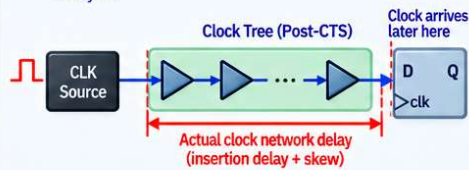
3 Difference between ideal and propagated clock

Feature	Ideal Clock (Pre-CTS)	Propagated Clock
Clock network delay	Assumed 0 (ideal)	Actual delays
Skew	0	Real (non-zero)
Insertion delay	0	Real (non-zero)
Clock waveform	Perfect (no distortion)	May have distortion (due to real network)
Analysis accuracy	Optimistic	Realistic
Used for	Early analysis, synthesis	Post-CTS analysis

Ideal clock is an assumption. Propagated clock uses the real clock.

4 How does STA use actual clock tree delay?

- After `set_propagated_clock`, STA reads the actual delays of the clock network from the post-CTS netlist.
- It computes the clock arrival time at each register including insertion delay and skew.
- These real arrival times are used for setup and hold analysis.



5 What happens if propagated clock is missing?

- The tool will continue to use an ideal clock (zero delay), ignoring the real clock network delays.
- This leads to optimistic timing results (setup/hold may look better than reality).
- Real timing issues (due to insertion delay, skew, etc.) may be missed.
- Can cause functional failures in silicon.



Risk:

- Missed setup/hold violations
- Overly optimistic timing
- Late ECOs and silicon respins

No propagated clock
= Blind analysis!

6 Why is this important for signoff?

- Signoff requires accurate and realistic timing
- `set_propagated_clock` ensures the tool use clock network with actual delays, skew, and
- It helps catch real timing violations before t
- Essential for signoff checks like:
 - Setup and hold timing
 - ECO closure
 - Multi-corner, multi-mode (MCM) anal
 - Signoff with extracted netlist (SPEF/DE

Signoff is about silicon confidence.
Use propagated clocks to see the real timing p



Key Takeaway:

Use `set_propagated_clock` after CTS to enable realistic timing analysis with actual clock tree delays. It is essential for accurate results and a successful signoff.

Real Clocks. Real
Real Silicon Su

Inputs Don't
Arrive Instantly!
Tell STA When They Arrive!

Input Delay

Model External Data Timing – So STA Can See the Real World!

Correct Input
= Accurate
= Reliable C

1 What is set_input_delay?

- `set_input_delay` is an SDC command used to specify the arrival time of input data relative to a clock.
- It tells the STA tool when the data launched from an external source (e.g., another chip) will arrive at the input port of your design.
- It is used only for primary input ports (not internal signals).

SDC Example:

```
set_input_delay -max 2.0 -clock clk [get_ports data_in]
set_input_delay -min 0.5 -clock clk [get_ports data_in]
```

It models the external world, not the internal design.

2 Why does an input need timing information?

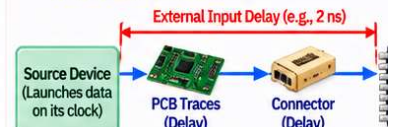
- The input data comes from an external device, which has its own clock and delays.
- Without input delay, STA assumes the data arrives at time 0, which is unrealistic and can give overly optimistic results.
- Providing input delay allows STA to correctly analyze setup and hold timing at the input registers.



Input delay models the real arrival time of external data.

3 What is external input delay?

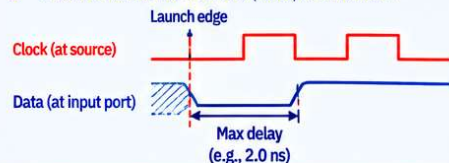
- External input delay is the time taken by the data from the launching clock edge at the source to the input of your chip.
- It includes delays from the external device, PCB routing, connectors, and any other external components.
- It is outside the chip and must be estimated or provided by the system/board team.



External input delay = device delay + board delay + connector delay

4 What is -max input delay?

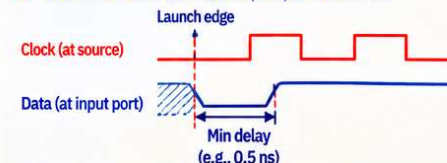
- `-max input delay` specifies the latest (maximum) arrival time of the input data at the chip relative to the clock edge.
- Used for setup timing analysis.
- It tells STA the worst-case (slow) arrival time.



`-max input delay` is used for setup checks (latest arrival time).

5 What is -min input delay?

- `-min input delay` specifies the earliest (minimum) arrival time of the input data at the chip relative to the clock edge.
- Used for hold timing analysis.
- It tells STA the best-case (fast) arrival time.



`-min input delay` is used for hold checks (earliest arrival time).

6 How does input delay affect setup and hold timing?

- A larger `-max input delay` reduces the available setup, making setup timing harder.
- A smaller `-min input delay` reduces the time to clock edge and data arrival, making hold timing harder.
- Correct input delays ensure STA checks the timing window and avoids optimistic results.

Effect of Input Delay on Timing

Input Delay	Setup Timing	Hold Timing
Larger -max	Harder (less time)	No direct effect
Smaller -max	Easier (more time)	No direct effect
Smaller -min	No direct effect	Harder
Larger -min	No direct effect	Easier



Key Takeaway:

Use `set_input_delay` to model when external data arrives at your chip input ports. Provide both `-max` and `-min` in values to get accurate setup and hold analysis. Correct input delays lead to realistic timing and a robust design.

Good Constr
Make Real

Your Chip
Doesn't Work Alone!
Tell STA About the Outside World!

Output Delay

Model the External World – Ensure Correct Timing at the Chip Output!

Correct Output
= Reliable Inte
= Successful S

1 What is set_output_delay?

- `set_output_delay` is an SDC command used to specify the required arrival time of output data at the external receiver relative to a clock.
- It tells the STA tool how much time is available for the data to travel from your chip's output port to the receiving device.
- It is used only for primary output ports (not internal signals).

SDC Example:

```
set_output_delay -max 2.0 -clock clk [get_ports data_out]
set_output_delay -min 0.5 -clock clk [get_ports data_out]
```

2 Why does an output need timing information?

- The output data from your chip goes to an external device (e.g., another chip, FPGA, memory).
- The external device has its own setup and hold requirements.
- STA needs this information to ensure that the data arrives at the receiver in time (setup) and does not change too early (hold).
- Without output delay, STA will not know the timing requirements at the other end, leading to unrealistic results.



3 What is external output timing?

- External output timing is the time budget from your output port to the input of the receiving device.
- It includes delays from package, PCB routing, connectors, and any other external components.
- It also accounts for the setup and hold requirements of the receiving device.
- This information is typically provided by the system team or the receiving device datasheet.

Example of External Output Timing:



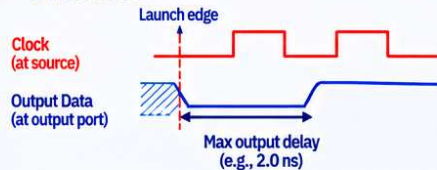
It models the external world, not the internal design.

Output delay tells STA the required arrival time at the receiving device.

Total External Output Delay (e.g., 2.0 ns)

4 What is -max output delay?

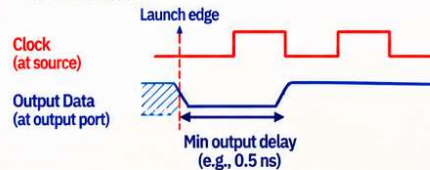
- max output delay specifies the latest (maximum) required arrival time of the output data at the receiving device relative to the clock edge.
- Used for setup timing analysis.
- It tells STA the worst-case (slow) arrival time allowed at the receiver.



-max output delay is used for setup checks (latest arrival time at receiver).

5 What is -min output delay?

- min output delay specifies the earliest (minimum) required arrival time of the output data at the receiving device relative to the clock edge.
- Used for hold timing analysis.
- It tells STA the best-case (fast) arrival time allowed at the receiver.



-min output delay is used for hold checks (earliest arrival time at receiver).

6 How does output delay affect timing budget?

- Output delay reduces the available time for data inside your chip (from the launching flip-flop to the receiving flip-flop).
- A larger -max output delay makes setup timing more difficult (more time), while a smaller -max makes it harder.
- A smaller -min output delay makes hold timing more difficult (less time), while a larger -min makes it harder.
- Incorrect output delays can lead to optimistic or pessimistic results, causing functional failures in the real system.

Effect of Output Delay on Timing		
Condition	Setup Timing	Hold Timing
Larger -max	Easier (more time)	No direct effect
Smaller -max	Harder (less time)	No direct effect
Smaller -min	No direct effect	Easier (more time)
Larger -min	No direct effect	Harder (less time)



Key Takeaway:

Use `set_output_delay` to model the required arrival time of your chip's output data at the receiving device. Provide both -max and -min values to get accurate setup and hold analysis and ensure reliable system-level timing.

Right Constraints
Enable Real Product

Chips Don't Work Alone
They Talk to Other Chips!

Input and Output Timing Environment

A Chip Works with the Outside World – Model the Complete Path for Accurate Timing!

Correct I/O Constraints
= Reliable Silicon
= Successful Product

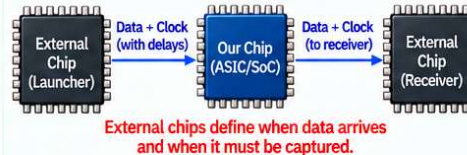
1 Why does a chip not exist in isolation?

- A chip is part of a larger system (SoC, board, module, product).
- It communicates with other chips (CPU, memory, PHY, sensors, etc.) through I/O pins.
- The signals travel outside the chip through PCB traces, connectors, and sometimes cables before reaching another device.
- Therefore, timing depends on both internal logic and the external environment.

A chip only works correctly when its external connections are also considered.

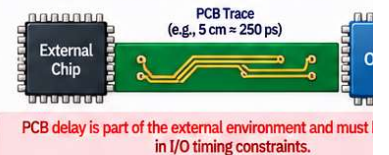
2 How do external chips affect I/O timing?

- The external device has its own clock, internal delays, and setup/hold requirements.
- For inputs: the launching device's clock and delay define when data arrives at your chip.
- For outputs: the receiving device's setup/hold requirements define how fast/slow your chip must drive data.
- Mismatches between clocks, voltages, and interface standards (e.g., DDR, SPI, I²C) also affect timing.



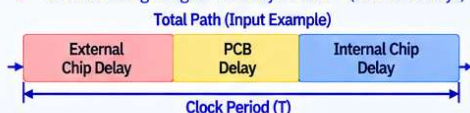
3 How does PCB delay affect constraints?

- Signals travel on PCB traces, which introduces delay (depends on trace length, material, and routing).
- PCB delay reduces the time available for internal logic to compute input delay (`set_input_delay`).
- For inputs: PCB delay is added to the external delay (`set_input_delay`).
- For outputs: PCB delay is part of the external delay (`set_output_delay`).
- Longer traces, vias, and connectors can significantly increase delay (tens to hundreds of ps).



4 How is internal timing budget calculated?

- Total timing = External device delay + PCB delay + Internal chip delay.
- For input paths (setup):
Required time at input = $T_{clk} + T_{launch_ext} + T_{pcb} - T_{setup_int}$
- For output paths (setup):
Available time for internal logic = $T_{clk} - (T_{setup_ext} + T_{pcb})$
- Internal timing budget = Total cycle time - (external delays).



5 What happens if I/O delay is missing?

- STA will assume 0 ns external delay by default (unrealistic).
- Input data may appear to arrive earlier → optimistic setup results, missed violations.
- Output data may appear to have more time → optimistic results, failures in real hardware.
- Can lead to silicon bring-up issues, functional failures, and re-spins.



Without proper I/O constraints:

- Timing looks better than reality.
- Violations may be missed.
- Chip may fail in system even if STA shows clean.
- Debug and re-spin cost increases significantly.

Missing I/O delay = False Confidence = High Risk!

6 Who provides I/O timing values?

- System architect / SoC team – overall system requirements.
- Board team (PCB designers) – PCB trace delays, routing information.
- Device datasheets – external chip timing (e.g., PHY, sensors).
- Often provided as an Interface Control Document or timing spec from the customer or system team.

Typical Sources of I/O Timing Values			
System Team (Requirements)	Datasheet (External Device)	PCB Team (Trace Delays)	Customer (Timing Spec)

PD engineers use these values to create realistic `set_input_delay` and `set_output_delay` constraints.



Key Takeaway:

A chip works in a system. Always include external device delays and PCB delays in I/O constraints using `set_input_delay` and `set_output_delay`. Accurate I/O timing ensures correct STA results, reliable silicon, and successful system integration.

Think System
Not Just Chip

Timing is not Enough!
Also Respect
Electrical Limits!

SDC Design Rule Constraints

Constrain for Real Silicon – Meet Electrical Limits, Build a Clean and Reliable Design!

Clean Constraints
= Clean Design
= Successful Product

1 What are design rule constraints in SDC?

- Design rule constraints (DRCs/DRVs) in SDC are constraints that specify electrical and structural limits for the synthesized design.
- They include limits on transition, capacitance, fanout, and sometimes load.
- These constraints guide synthesis to generate a design that is not only timing clean but also physically realizable and manufacturable.
- Common commands: `set_max_transition`, `set_max_capacitance`, `set_max_fanout`, `set_load`.

2 How are SDC DRVs different from physical DRC?

- SDC DRVs (Design Rule Violations) are checked during synthesis/STA and are logical/electrical constraints (e.g., transition, capacitance, fanout).
- Physical DRCs are checked during physical design (layout) and are related to geometric rules (e.g., spacing, width, enclosure).
- SDC DRVs ensure good electrical behavior and synthesizable design quality, while physical DRCs ensure manufacturability at the layout level.

SDC DRVs (Logical/Electrical)

- Checked during synthesis/STA
- Transition, capacitance, fanout

Physical DRCs (Geometric)

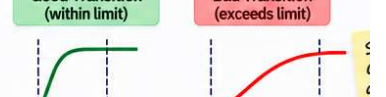
- Checked during layout (P&R)
- Spacing, width, enclosure

3 What is max transition?

- Max transition specifies the maximum allowed slew (rise/fall time) of a signal at a pin.
- If the transition is too slow, it can cause excessive delay, increase power, and may violate cell setup/hold times.
- It is specified using: `set_max_transition <value>`

Good Transition (within limit)

Bad Transition (exceeds limit)



Design rule constraints ensure the design meets electrical limits and can be implemented successfully in silicon.

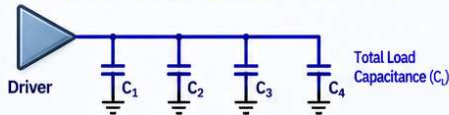
- Transition, capacitance, fanout
- Electrical/timing related
- Specified in SDC
- Ensures good design quality

- Spacing, width, enclosure
- Geometric/layout related
- Defined in PDK (tech file)
- Ensures manufacturability



4 What is max capacitance?

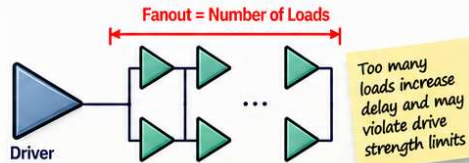
- Max capacitance specifies the maximum allowed load capacitance on a pin/net.
- If the load is too high, the gate may not be able to drive it properly, causing large delay and poor signal integrity.
- It is specified using: `set_max_capacitance <value>`



Keep total load capacitance within the specified limit to ensure proper drive strength and timing.

5 What is max fanout?

- Max fanout specifies the maximum number of loads that can be driven by a single output.
- If a net drives too many loads, it increases delay, power, and can violate cell drive strength limits!
- It is specified using: `set_max_fanout <value>`



6 Why must synthesis obey electrical limits?

- Synthesis must obey electrical limits to ensure it is physically realizable and works reliably in silicon.
- Violating transition, capacitance, or fanout limits:
 - Excessive delay and timing failures
 - Increased power consumption
 - Signal integrity issues (noise, ringing)
 - Reliability problems
 - Inability to meet library/technology specifications
 - Late-stage fixes during physical design

By obeying electrical limits, synthesis produces a robust, and implementable design, reducing re-spin cost.



Key Takeaway:

Use SDC design rule constraints (`set_max_transition`, `set_max_capacitance`, `set_max_fanout`) to guide synthesis within electrical limits. These constraints complement timing constraints and help ensure a reliable and manufacturable silicon.

Constrain T
for a Better T

A Clean Edge
Drives a Better Chip!

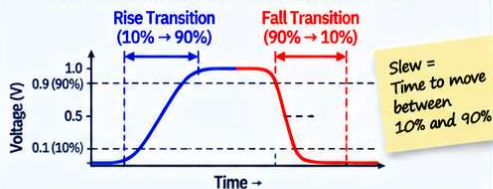
set_max_transition

Control Signal Slew – For a Clean, Reliable, and High-Performance Design!

Control S
= Avoid Vi
= Ensure I

1 What is transition or slew?

- Transition (slew) is the time taken for a signal to rise from 10% to 90% (rise time) or fall from 90% to 10% (fall time) of its voltage level.
- It indicates how fast the signal changes at a pin.
- It is measured in seconds (ns or ps).
- A smaller transition means a sharper (faster) edge, while a larger transition means a slower edge.



2 What does set_max_transition do?

- `set_max_transition` specifies the maximum allowed transition (slew) for a signal at a pin.
- It is a design rule constraint used in synthesis and STA to ensure the signal transitions are not too slow.
- It helps the tool size cells, buffer nets, or optimize the design to meet the transition limit.

SDC Example:

```
set_max_transition 0.1 [get_ports clk]
set_max_transition 0.2 [get_ports data_in]
set_max_transition 0.2 [get_nets *]
```

This tells the tool that the transition at the specified pins or nets must not exceed the given value (in ns).

3 Why is excessive slew dangerous?

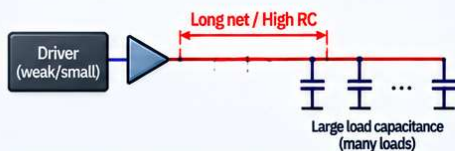
- Large slew increases dynamic power consumption (short-circuit power).
- It can cause timing violations due to increased delays in subsequent cells.
- It reduces noise margin and signal integrity.
- It can lead to functional failures in advanced nodes (e.g., glitching, false switching).
- It violates library and technology specifications.

Effect of Excessive Slew:



4 What causes transition violations?

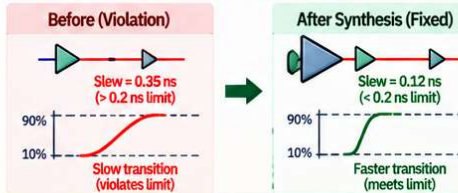
- Insufficient cell drive strength (small cells).
- High load capacitance (large fanout or long nets).
- Long interconnects with high RC.
- Multiple loads on a single net.
- Constraints too tight for the given design.
- Suboptimal synthesis settings.



More load and resistance = slower transition!

5 How does synthesis fix transition violations?

- Uses larger drive strength cells.
- Inserts buffer/inverter cells.
- Re-sizes gates on critical nets.
- Optimizes logic structure (e.g., upsizing, replication).
- May perform fanout splitting or reduce logic depth.



Synthesis optimizes the design to meet the `max_transition` constraint.

6 How does STA check transition violations?

- STA calculates the transition at each pin using a timing model and the actual net load (capacitance).
- It compares the calculated slew with the `set_max_transition` value.
- If the calculated transition exceeds the limit, a Design Rule Violation (DRV) is reported.
- Example STA report (simplified):

```
** Design Rule Violation: Transition
Net      : data_out
Pin      : U1/Y
Actual transition : 0.35 ns
Limit     : 0.20 ns
Slack     : -0.15 ns (VIOLATION)
```

STA ensures all signal transitions meet the specified limits for reliable timing and electrical behavior.



Key Takeaway:

Use `set_max_transition` to control signal slew, avoid design rule violations, and ensure reliable, low-power, high-performance silicon. A clean transition leads to better timing, lower power, and a robust design.

Right
for a Bright

Large Loads
Slow Down Your Chip!

set_max_capacitance

Keep the Load in Check – For Faster, Cleaner, and Reliable Designs!

Control Cap
= Lower D
= Better T
= Reliable

1 What is capacitance?

- Capacitance is the ability of a node to store electrical charge.
- In digital circuits, it represents the total load seen by a driver (e.g., gates, interconnects, pins, etc.).
- It is measured in Farads (F), typically in femtofarads (fF) or picofarads (pF).
- A higher capacitance means more charge is required to switch the node, which can increase delay.



2 What does set_max_capacitance do?

- `set_max_capacitance` specifies the maximum allowed load capacitance at a pin, net, or port.
- It is a design rule constraint used in synthesis and STA.
- It ensures the synthesized design does not drive excessive capacitive loads.
- Helps maintain good timing, signal integrity, and drive strength.

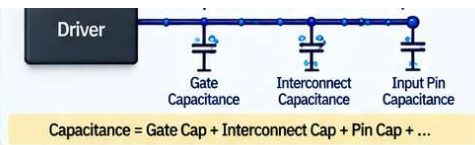
SDC Example:

```
set_max_capacitance 0.5 [get_ports clk]
```

3 Why does high capacitance increase delay?

- A higher capacitance requires more charge to switch the node ($Q = C \times V$).
- The driver needs more time to charge and discharge the larger load.
- This increases the RC (resistance $\times$ capacitance) leading to a larger propagation delay.
- Higher capacitance also worsens the signal transition.

Small Capacitance (Faster) Large Capacitance (Slower)



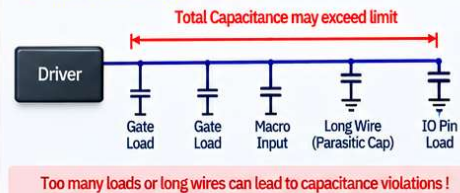
```
set_max_capacitance 0.2 [get_ports data_out]
set_max_capacitance 0.3 [get_nets *]
```

This tells the tool that the total capacitance at the specified pins or nets must not exceed the given value (in pF).



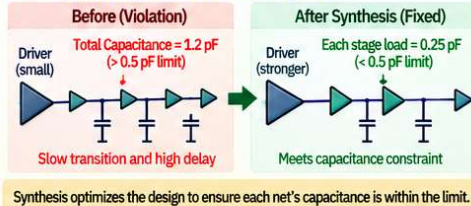
4 What causes capacitance violations?

- Large fanout (driving too many gates).
- Long interconnect wires with high parasitic capacitance.
- Large input pins or macros.
- Bus connections with multiple loads.
- Poor synthesis settings (insufficient buffering).
- Unoptimized logic structure.



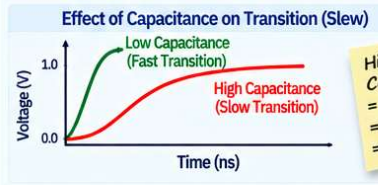
5 How does synthesis fix max capacitance violations?

- Inserts buffer/inverter cells to reduce the load per stage (buffer splitting).
- Upsizes the driver cell to a stronger drive strength.
- Re-structures the logic (e.g., logic duplication, partitioning).
- Optimizes the netlist to reduce fanout.
- May use dedicated buffers for long nets.



6 How does capacitance affect transition time?

- A higher capacitance increases the RC time constant, which makes the signal transition (slew) slower.
- Slower transitions can cause setup/hold violations, increase dynamic power, and may violate maximum fanout constraints.
- Controlling capacitance helps keep the slew within limits and ensures reliable timing.



Key Takeaway: Use `set_max_capacitance` to control the load seen by your design. Keep capacitance within limits to ensure lower delay, good transition, and a reliable, manufacturable design.

Control the load for a faster, cleaner design.

set_max_fanout

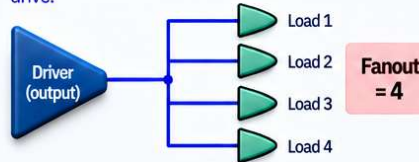
Limit the Load Count – Prevent Congestion, Ensure Timing and Signal Integrity!

One Driver, Many Loads! Keep it Under Control!

Reasonable Fanout = Better Timing = Clean Design = Successful Project

1 What is fanout?

- Fanout is the number of loads (inputs) driven by a single output (net).
- Each load can be a gate input, flip-flop input, macro input, or an output port.
- It indicates how many places the signal needs to drive.



Fanout = Number of loads driven by a single output.

2 What does set_max_fanout do?

- `set_max_fanout` specifies the maximum number of loads that a single output can drive.
- It is a design rule constraint used in synthesis and STA.
- It ensures the synthesized netlist does not exceed the given fanout limit.
- It helps maintain good timing, signal integrity, and drive strength.

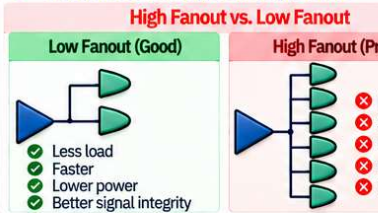
SDC Example:

```
set_max_fanout 4 [get_ports clk]
set_max_fanout 8 [get_ports data_out]
set_max_fanout 6 [get_nets *]
```

This tells the tool that no single output should drive more than the specified number of loads.

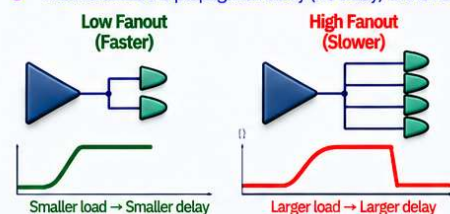
3 Why does high fanout cause problems?

- A driver has limited drive strength. Driving many loads increases the load capacitance.
- It can violate timing (setup/hold), transition, and even functional correctness.
- It increases power consumption.
- It can cause signal integrity issues (noise, IR drop).
- It may exceed library/technology limits.



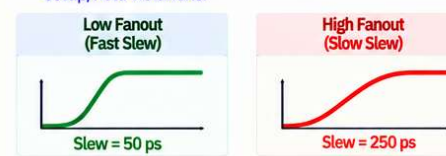
4 How does high fanout affect delay?

- More loads → higher total capacitance on the net.
- The driver has to charge/discharge a larger capacitance.
- This increases the propagation delay (RC delay) of the net.



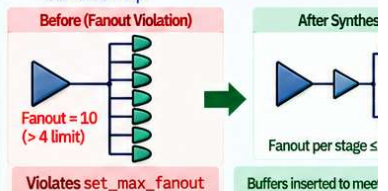
5 How does high fanout affect slew?

- More loads → higher capacitance.
- A larger capacitance takes more time to charge and discharge.
- This results in a slower signal transition (higher slew).
- Higher slew can further increase delay and cause setup/hold violations.



6 How is fanout violation fixed?

- Synthesis uses buffer insertion to split the load.
- It can resize the driver to a stronger cell (if available).
- It may restructure the logic (e.g., insert intermediate buffers or create a buffer tree).
- Manual RTL optimization (reduce unnecessary logic) can also help.



Key Takeaway: Use `set_max_fanout` to limit the number of loads driven by a single output. Controlling fanout helps reduce delay and slew, ensures reliable timing, lower power, and a clean, manufacturable design.

Control Fanout for a faster, cleaner design.

Output Load and Driving Cell

Model the External Environment – Get Realistic Timing and a Robust Design!

Model the Real World for Better Design!

Accurate Modeling = Better Simulation = Reliable Design

1 What is set_load?

- `set_load` specifies the external load capacitance seen at an output port or pin.
- It tells the tool how much capacitive load (in pF or fF) is connected to the output.
- It is used for output ports (typically to other chips, PCB, or off-chip devices).
- It helps the synthesis tool size the output drivers correctly.

2 Why must output load be modeled?

- The output drives external circuits (PCB traces, another chip, connectors, etc.) which add capacitance.
- The tool must know this load to estimate delay, slew, and required drive strength.
- Without a proper load, synthesis may under-size drivers, causing high delay and slew violations.
- Accurate load modeling ensures correct timing, power, and signal integrity.

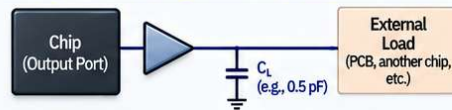
3 What is set_driving_cell?

- `set_driving_cell` specifies the reference cell (output pin) that drives an input port or pin.
- It models the drive strength (output resistance, transition) of the external source.
- It helps the tool estimate the input signal transition more realistically.
- It is specified as: `set_driving_cell <lib_cell> [get_ports <port>]`

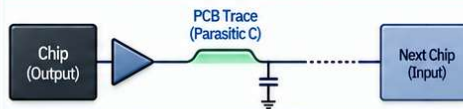
- It is specified as: `set_load <value> [get_ports <port>]`

SDC Example:

```
set_load 0.5 [get_ports data_out]
```



power, and signal integrity.

Example Output Environment:

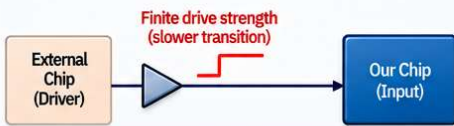
Real outputs always see some capacitance in the real world!

SDC Example:

```
set_driving_cell -lib_cell INV_X2 -pin Z [get_ports Z]
```

**4 Why must input drive strength be modeled?**

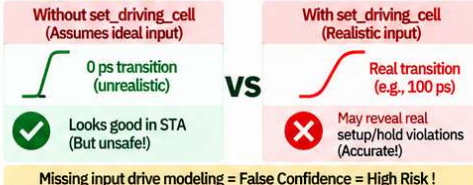
- Input signals come from external sources (another chip, board, etc.) with finite drive strength.
- The input transition time (slew) depends on the external driver's output resistance and the line capacitance.
- Accurate modeling helps the tool estimate correct transition, delay, and setup/hold timing.
- It avoids optimistic timing assumptions.



Inputs are not ideal – model the real driver!

5 What happens if input driving strength is missing?

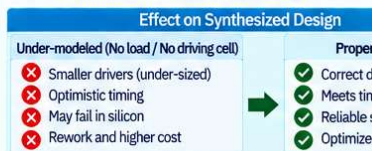
- The tool assumes an ideal (zero) input transition (by default).
- This is overly optimistic and gives unrealistically good timing results.
- It can hide setup/hold violations and lead to silicon failures.
- The synthesized design may be under-designed for real input conditions.



Missing input drive modeling = False Confidence = High Risk!

6 How do load and driving cell affect

- Output load (set_load):** Tool sizes the output delay and slew with the given load.
- Input driving cell (set_driving_cell):** Tool uses transition to meet setup/hold timing.
- They directly impact cell sizing, buffer insert timing optimization.
- Accurate modeling leads to better area, power and a robust, manufacturable design.

**Key Takeaway:**

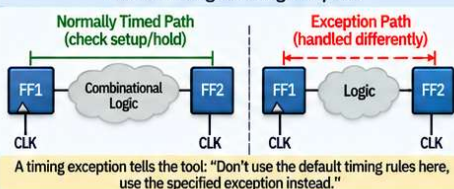
Use `set_load` to model the external output load and `set_driving_cell` to model the input drive strength. Accurate modeling of the I/O environment helps synthesis create a reliable, high-performance, and silicon-proven design.

Real Enviro
Build RFocus on
What Really Matters!**Timing Exceptions**

Not All Paths Are Equal – Use Exceptions for a Correct and Realistic Design!

Right Exce
= Reliable
= Successf**1 What is a timing exception?**

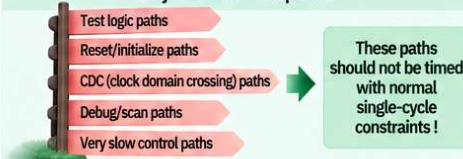
- A timing exception is a constraint that tells the STA tool not to check a path with the normal default timing rules, or to use different timing rules.
- Examples: false path, multicycle path, max delay, min delay, etc.
- It is specified in the SDC using commands like `set_false_path`, `set_multicycle_path`, `set_max_delay`, etc.

Normal Timing vs Timing Exception

A timing exception tells the tool: "Don't use the default timing rules here, use the specified exception instead."

2 Why should not every path be timed normally?

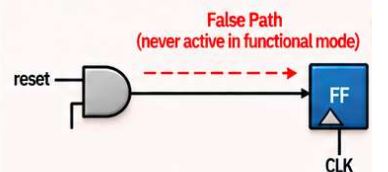
- Not all paths affect functional operation or performance.
- Some paths are logically disabled, used only during test/reset, or are very slow and do not need to meet single-cycle timing.
- Timing all paths equally can lead to unrealistic violations, larger design area, higher power, and unnecessary effort to fix non-critical paths.
- Exceptions help focus on real, functional, and performance-critical paths.

Why We Need Exceptions?

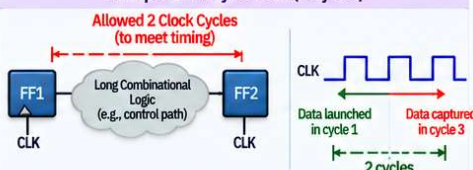
These paths should not be timed with normal single-cycle constraints!

3 What is a false path?

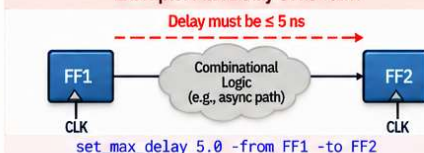
- A false path is a path that is logically or functionally never activated, so it does not need to be timed.
- Used for paths like: test logic, reset paths, reset exclusive logic, unused logic, etc.
- Specified using: `set_false_path`

Example: False Path (Reset Logic)**4 What is a multicycle path?**

- A multicycle path is a path that is allowed to take more than one clock cycle to propagate data.
- Used for paths with long latency, low-speed control logic, or specific design architecture.
- Specified using: `set_multicycle_path`

Example: Multicycle Path (2 Cycles)**5 What is a max delay constraint?**

- A max delay constraint specifies an upper limit on the propagation delay between two points (startpoint and endpoint).
- It is used for non-critical paths or special timing requirements (e.g., IO paths, asynchronous paths).
- Specified using: `set_max_delay`

Example: Max Delay Constraint**6 Why can wrong exceptions cause a silicon failure?**

- If a real functional path is incorrectly excluded as false or given too many cycles, the tool will miss timing violations that go unnoticed.
- This can lead to data corruption, functional failure, or silicon not working at the desired frequency.
- Therefore, exceptions must be used carefully and thoroughly (with design intent, simulation, and silicon verification).

Risk of Wrong Exceptions

- Real paths may be ignored by STA.
- Timing violations are not reported.
- Can lead to functional errors in silicon.
- Difficult to debug after fabrication.
- May result in respin (high cost and time).



Wrong exceptions can turn a good design into a failure.

**Key Takeaway:**

Use timing exceptions (`set_false_path`, `set_multicycle_path`, `set_max_delay`) where appropriate to focus on real timing paths. Apply exceptions carefully and verify them, because incorrect exceptions can lead to silicon failure.

Right Constr
Bring the RightNot All Paths
Need to be Timed!**set_false_path**

Tell the Tool What Not to Time – Focus on Real Paths, Get a Better Design!

Correct Ex
= Cleaner
= Better**1 What is set_false_path?**

- `set_false_path` is a timing exception that tells the STA tool to ignore timing analysis on a specified path because it is logically or functionally impossible to be activated.
- It is used for paths that are never exercised in normal operation, such as: reset paths, test logic, mutually exclusive logic, etc.

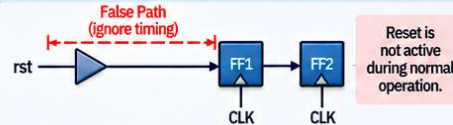
2 What does -from mean?

- `-from` specifies the starting point(s) of the false path.
- It can be a port, pin, clock, cell, or any valid collection.
- All timing paths starting from the specified `-from` objects will be ignored (to the specified `-to` or `-through`, if given).

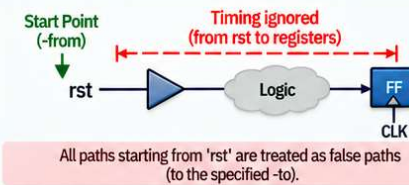
3 What does -to mean?

- `-to` specifies the endpoint(s) of the false path.
- It can be a port, pin, register, cell, or any valid collection.
- All timing paths ending at the specified `-to` (from the specified `-from`, if given) will be ignored.

- It helps reduce unnecessary timing violations and focuses the tool on real, functional paths.
- Example: `set_false_path -from [get_ports rst] -to [get_registers *]`

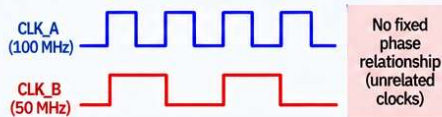
Example: Reset Path as False Path**Example: -from (Starting Point)**

```
set_false_path -from [get_ports rst] -to [get_registers *]
```



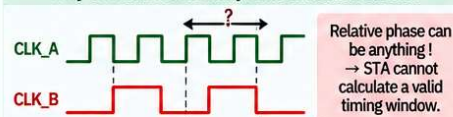
- They can run at different frequencies and are independent of each other.
- Examples: CPU clock (100 MHz) and peripheral clock (50 MHz), or clocks from different PLLs.
- Such clocks are called "unrelated" clocks.

Example: Two Asynchronous Clocks



- For unrelated clocks, the relative phase can be anything, so STA cannot determine a meaningful worst-case timing.
- If timed normally, it may report huge and meaningless violations.
- Therefore, we tell the tool to ignore these paths using clock groups.

Why STA Cannot Normally Time Unrelated Clocks?



- It tells STA to ignore timing paths between the clock groups.
- This is a cleaner and safer way to exclude all paths between clock domains.

SDC Example

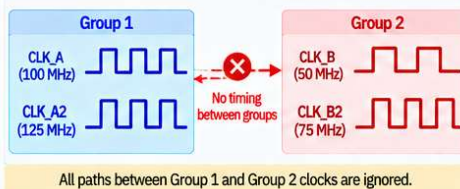
```
set_clock_groups -asynchronous \
                  -group {CLK_A} \
                  -group {CLK_B}
```

This tells STA to ignore all timing paths between CLK_A

4 What is an asynchronous clock group?

- A clock group is a collection of clocks defined together in set_clock_groups.
- An asynchronous clock group means the clocks in one group are asynchronous (unrelated) to the clocks in another group.
- All paths between clocks in different groups are ignored in timing analysis.

Example: Asynchronous Clock Groups



5 How is set_clock_groups different from set_false_path?

- set_clock_groups is used to exclude all paths between two (or more) asynchronous clock domains.
- set_false_path is used to exclude specific paths (defined by -from, -to, -through).
- set_clock_groups is more scalable and preferred for unrelated clocks.
- set_false_path is more manual and can miss some paths.

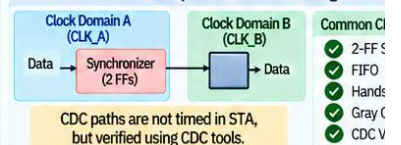
set_clock_groups vs set_false_path

Feature	set_clock_groups	set_false_path
Purpose	Exclude all paths between clock groups	Exclude specific paths
Scope	All paths between clocks in the groups	Only paths defined by -from, -to, -through
Typical Use	Unrelated/asynchronous clocks	Specific non-functional paths (reset, scan, etc.)
Ease of Use	Simple and scalable	Manual, error-prone for large designs

6 How are CDC paths handled?

- CDC (Clock Domain Crossing) paths transfer data between asynchronous clock domains.
- These paths are functionally important, so they simply ignored.
- They must be handled using proper CDC design (e.g., synchronizers, FIFOs, handshakes, gray code).
- Static CDC tools are used to check these paths for correctness (not STA).
- In STA, these paths are usually excluded using set_clock_groups, and separately verified using CDC tools.

Example: CDC Path Handling



Key Takeaway:

Use set_clock_groups to tell STA which clocks are asynchronous and avoid meaningless timing violations. Handle CDC paths with proper design techniques and dedicated CDC verification tools.

Different Clock Domains
Smarter Constraints
Reliable CDC

Not Every Path Needs 1 Cycle!

Multicycle Paths

Give More Time to the Paths That Need It – Correct Constraints, Better Designs!

Realistic Constraints
= Better Constraints
= Reliable Designs

1 What is set_multicycle_path?

- set_multicycle_path is an SDC command used to allow more than one clock cycle for a path to meet timing.
- It changes the required time for setup and/or hold checks on the specified path.
- It is used when a path is functionally designed to take multiple cycles (not a single cycle).
- It is specified using options like -setup and -hold with -from, -to, or -through.

SDC Example

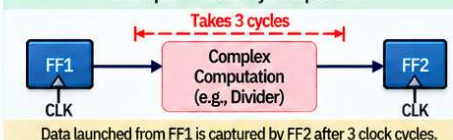
```
# Allow 3 cycles for setup
set_multicycle_path 3 -setup -from [get_cells U1] -to [get_cells U2]

# Adjust hold (N-1)
set_multicycle_path 2 -hold -from [get_cells U1] -to [get_cells U2]
```

2 Why does a multicycle path exist?

- Some operations naturally take multiple clock cycles to complete (e.g., complex arithmetic, FSM paths, processor pipelines).
- Not all data is required in the very next cycle.
- Using multicycle paths avoids unrealistic timing violations and unnecessary over-optimization.
- It helps match the design's functional behavior and improves QoR (area, power, performance).

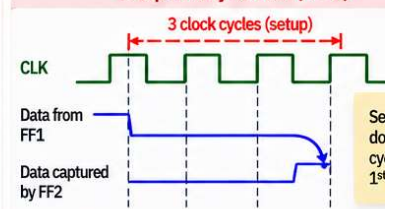
Example: A Multi-Cycle Operation



3 What is a setup multicycle path?

- A setup multicycle path (N) gives N clock cycles to arrive at the endpoint (instead of the default 1-cycle requirement).
- The setup check is performed N cycles later.
- Specified using: set_multicycle_path N -setup

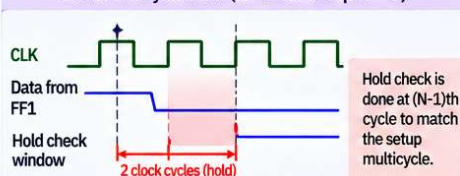
Setup Multicycle Path (N = 3)



4 What is a hold multicycle path?

- A hold multicycle path (N) changes the hold check to N clock cycles (typically N-1 when setup is N).
- It ensures that the data is not required to be stable immediately after the launch edge.
- Specified using: set_multicycle_path N -hold ...

Hold Multicycle Path (N = 2 for setup N = 3)



5 Why are setup and hold values related?

- If setup is N, hold is usually N-1.
- This maintains the correct timing window:
 - Setup check ensures data is captured after N cycles.
 - Hold check ensures data is not captured too early (within the first N-1 cycles).
- Using consistent values avoids false hold violations and ensures correct functionality.

Timing Window for Multicycle Path (N = 3)



6 What happens if a multicycle path is not used?

- The tool assumes a default 1-cycle timing requirement.
- This can lead to setup timing violations because the path cannot meet the 1-cycle requirement.
- Designers may unnecessarily add buffers or resize logic, increasing area and power.
- It can also lead to incorrect optimization and a suboptimal design.
- In worst cases, the design intent is not met and the function correctly if real multi-cycle behavior is expected.

With vs Without Multicycle Constraint

Without Multicycle Path	With Correct Multicycle Path
✗ Setup timing violations	✓ No false timing violations
✗ Unnecessary optimization (area/power overhead)	✓ Accurate optimization
✗ Poor QoR	✓ Better area, power, and performance
✗ May hide real design intent	✓ Matches functional intent
	✓ Reliable silicon



Key Takeaway:

Use set_multicycle_path for paths that are intentionally designed to take multiple cycles. Always specify both -setup and -hold (usually N and N-1) and ensure the constraint matches the design intent.

Right Constraints
Enable Smarter Designs

More Control Over Critical Paths!

set_max_delay and set_min_delay

Define Your Own Timing Requirements – When Clock Timing is Not Enough!

Custom Constraints
= Better Designs
= Reliable

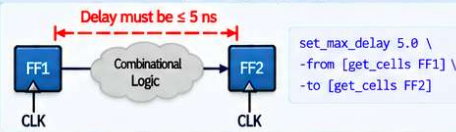
1 What is set_max_delay?

2 What is set_min_delay?

3 When are they used?

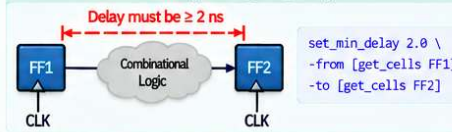
- `set_max_delay` specifies an upper limit on the propagation delay between two points (from $\rightarrow$ to).
- It is a custom timing constraint, independent of the clock period.
- It tells STA that the data must arrive within the specified time.
- It is typically used for special paths like I/O paths, asynchronous paths, or paths with tighter requirements than the clock period.

Example: `set_max_delay`



- `set_min_delay` specifies a lower limit on the propagation delay between two points (from $\rightarrow$ to).
- It ensures that the data does not arrive too early.
- It is used to model real-world delays (e.g., external devices, board delays) or to avoid hold time violations.
- It is also independent of the clock period.

Example: `set_min_delay`

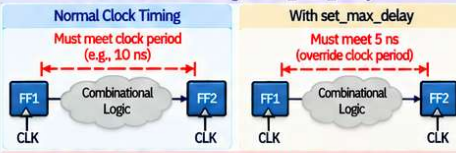


- Used when the normal clock-based timing is not or not applicable.
- Common use cases:
 - ▶ I/O paths (input to internal, internal to output)
 - ▶ Paths between asynchronous clocks
 - ▶ Paths through large logic where you want a tighter, more accurate delay model
 - ▶ Bus/interface timing requirements (e.g., protocol-specific delays)
 - ▶ Modeling external delays (board, package, other components)
 - ▶ Constraining specific paths (e.g., test paths, handshakes)

4 How do they override normal clock timing?

- Normally, STA uses the clock period and clock uncertainties to calculate required times for setup and hold.
- When you apply `set_max_delay` or `set_min_delay` on a path, STA ignores the normal clock-based requirement for that path and uses the specified delay instead.
- These constraints take precedence over the default clock timing for the specified path.

Normal Clock Timing vs. `set_max_delay`

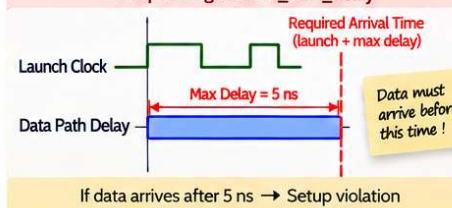


For this path, STA uses 5 ns instead of the 10 ns clock period.

5 How do they affect setup timing?

- `set_max_delay` directly sets the required time for setup check as (launch time + specified max delay).
- If the data arrives after this time, it is a setup violation.
- This can be tighter or looser than the normal clock period.

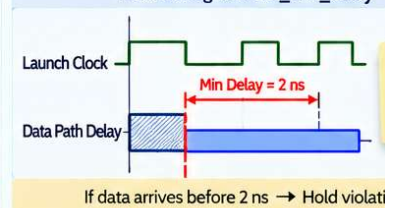
Setup Timing with `set_max_delay`



6 How do they affect hold timing?

- `set_min_delay` directly sets the required time for hold check as (launch time + specified min delay).
- If the data arrives before this time, it is a hold violation.
- This ensures the data is not too fast.

Hold Timing with `set_min_delay`



Key Takeaway:

- Use `set_max_delay` to set an upper limit (affects setup).
- Use `set_min_delay` to set a lower limit (affects hold).
- These constraints override the normal clock timing and are used for special cases like I/O, asynchronous paths, or protocol-specific requirements.

Right Constr
for Real-World

Different Modes
Different Realities
Use Case Analysis!

set_case_analysis

Tell the Tool What is Fixed in This Mode – Remove Inactive Logic, Get a Cleaner Design!

Constrain the
= Smaller Design
= Faster Analysis
= Better Results

1 What is `set_case_analysis`?

- `set_case_analysis` is an SDC command used to force a signal to a constant value (0 or 1) during timing analysis and synthesis.
- It tells the tool to assume that the signal always has that value in the specified mode.
- It is a mode constraint (not a timing constraint).
- Syntax:

```
set_case_analysis <value> <signal>
# Example
set_case_analysis 1 scan_en
```

2 Why is it used?

- Many signals are constant in a particular mode (e.g., test mode, low-power mode).
- Constraining these signals helps:
 - ▶ Remove inactive logic
 - ▶ Reduce design complexity
 - ▶ Improve timing analysis accuracy
 - ▶ Speed up synthesis and STA
 - ▶ Avoid false timing violations

Tell the tool
which mode
you are in!

3 How does it set a signal to 0 or 1?

- You specify the constant value (0 or 1) and the signal name in the SDC file.
- The tool treats the signal as tied to that constant value throughout the analysis.
- It is like virtually connecting the signal to VSS (0) or VDD (1) in that mode.

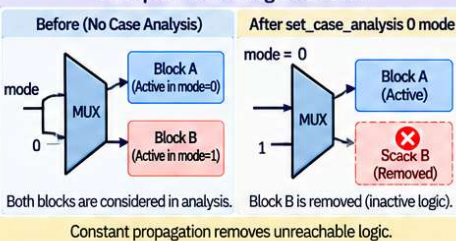
```
set_case_analysis 0 test_mode
set_case_analysis 1 scan_en
```



4 How does it remove inactive logic?

- When a signal is fixed to 0 or 1, the tool can determine which parts of the logic are unreachable.
- Logic driven by the constant value is simplified or removed (constant propagation).
- This reduces the design size and makes timing analysis faster and more accurate.

Example: Inactive Logic Removal

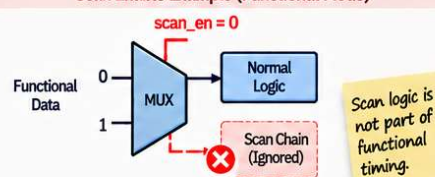


5 How is it used for scan enable?

- In functional mode, scan enable (`scan_en`) is always 0.
- We set it to 0 so that scan logic (muxes, scan chains) is ignored during timing analysis and synthesis.
- This prevents unnecessary timing paths through scan logic and reduces area/power.

```
# Functional mode (scan disabled)
set_case_analysis 0 scan_en
```

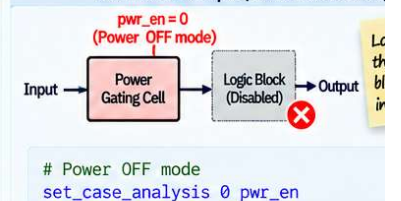
Scan Enable Example (Functional Mode)



6 How is it used for low-power?

- In low-power designs, some signals (e.g., power enables, isolation enables, retention controls) are active in specific modes.
- `set_case_analysis` is used to set these signals to known values in each power mode.
- This allows the tool to remove inactive power evaluation only the active part of the design.

Low-Power Example (Power Gated Block)



Key Takeaway:

- Use `set_case_analysis` to constrain constant signals in a specific mode.
- It helps remove inactive logic, improves timing accuracy, and reduces design complexity.
- Common uses: scan enable, test mode, low-power signals, configuration pins.

Right Mode
Right Analysis
Better Results

Design for Test
Design for Success!

Scan and DFT Constraints

Proper Constraints for Test Mode – A Clean Design in Both Functional and Scan Worlds!

Testable Design
= Higher Coverage
= Reliable

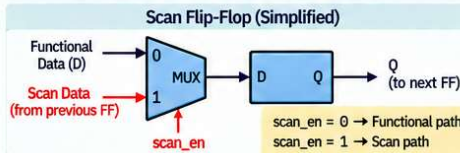
1 Why does scan require SDC constraints?

- Scan mode is a different operating mode than functional mode, with different timing behavior.
- Without proper constraints, STA and synthesis may treat scan paths as real functional paths, leading to wrong timing analysis and unnecessary optimization.
- SDC constraints tell the tools which paths to ignore, which signals are constant, and how to analyze the design correctly in each mode.

Correct scan constraints = clean timing reports, faster convergence, and reliable silicon.

2 What is scan enable?

- Scan enable (scan_en or test_mode) is a control signal that selects between functional mode and scan mode.
- When scan_en = 0 → design works in normal functional mode.
- When scan_en = 1 → scan flip-flops are connected in a shift register (scan chain) for test.
- It controls multiplexers (MUX) inside scan flip-flops to select between functional data and scan data.



3 Why use set_case_analysis for scan mode?

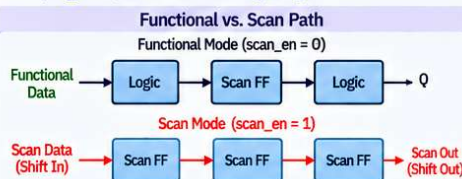
- In functional mode, scan_en is always 0, so it is inactive and should be removed from timing analysis.
- Using set_case_analysis 0 scan_en tells the tool to treat scan_en as a constant 0 in functional mode.
- This simplifies the design by removing inactive scan logic, reduces design complexity, and improves timing accuracy and synthesis results.

SDC Example (Functional Mode)

```
# scan_en is 0 in functional mode
set_case_analysis 0 scan_en
```

4 Why are scan paths different from functional paths?

- Functional paths carry real data during normal operation and must meet performance timing.
- Scan paths are used only during test mode (shift_in/shift_out) and are not active in functional mode.
- Scan paths can be much longer and do not need to meet the same timing requirements as functional paths.
- Therefore, scan paths are usually marked as false paths (or ignored) in functional timing analysis.



5 What false paths are used for scan?

- Typically, we use false paths to ignore scan paths in functional mode.
- Common scan false path constraints:
 - From scan input (scan_in) to scan output (scan_out)
 - Through scan enable (scan_en)
 - From test mode ports (e.g., scan_in, scan_en) to functional logic (if required)

SDC Examples for Scan False Paths

```
# Ignore all scan paths in functional mode
set_false_path -from [get_ports scan_in] -to [get_ports scan_out]

# Ignore paths through scan enable
set_false_path -through [get_ports scan_en]

# (Optional) Ignore paths from scan ports to functional logic
set_false_path -from [get_ports scan_in] -to [all_registers]
```

6 What happens if scan constraints are missing?

- STA will analyze scan paths as real functional paths, leading to many false timing violations.
- Tools may try to optimize long scan paths, increasing area and affecting functional design.
- Timing reports become noisy and hard to debug.
- It can hide real functional timing issues in the noise of scan-related violations.
- May lead to incorrect design changes, longer time, and even silicon failure if real issues are missed.

Example: Impact on Timing Report

Without Scan Constraints	With Proper Scan Constraints
✗ Thousands of timing violations on scan paths	✓ Scan paths ignored in functional mode
✗ Unnecessary optimization	✓ Clean and focused timing reports
✗ Large, noisy timing reports	✓ Better synthesis and timing closure
✗ Real issues may be hidden	✓ Easier sign-off and debugging



Key Takeaway:

- Use set_case_analysis to fix scan_en in functional mode and remove inactive scan logic.
- Use set_false_path to ignore scan paths during functional timing analysis.
- Proper scan constraints give clean reports, better QoR, and a reliable, testable design.

Test Today for a Better Tomorrow

Clock Gating
= Lower Power
= Longer Battery Life!

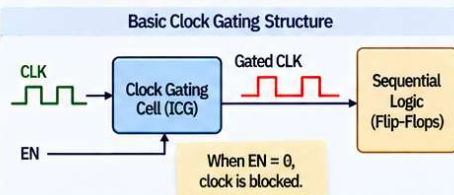
Clock Gating Constraints

Save Power with the Right Constraints – Model, Verify, Deliver!

Proper Constraints
= Correct Timing
= Real Power Savings
= Reliable Design

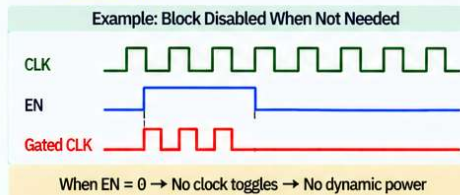
1 What is clock gating?

- Clock gating is a low-power technique where the clock signal to a block of sequential logic is disabled (gated) when it is not needed.
- A clock gating cell (ICG – Integrated Clock Gating) uses an enable signal to control the clock.
- When the enable is 0, the clock is blocked, and flip-flops in the block do not toggle.



2 Why is clock gating used?

- To reduce dynamic power consumption.
- To disable unnecessary switching in idle or inactive blocks.
- Commonly used in low-power designs like mobile, IoT, and battery-powered devices.
- Helps meet power targets without impacting functionality.



3 How does clock gating save power?

- Dynamic power in sequential logic is proportional to switching activity: $P_{dynamic} = \alpha \cdot C \cdot V^2 \cdot f$.
- Clock gating reduces the switching activity (stopping the clock).
- When the clock is disabled, flip-flops do not toggle, so internal combinational logic also does not toggle.
- This leads to significant savings, especially in blocks with many flip-flops.

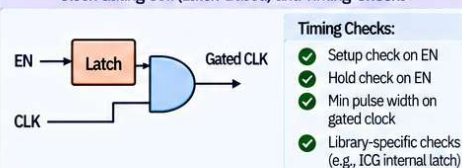
Power Comparison

Without Clock Gating (Always ON)	With Clock Gating
✗ Flip-flops toggle every cycle	✓ Flip-flops do not toggle when clock is gated
✗ Higher dynamic power	✓ Lower dynamic power

4 What timing checks exist around clock gating?

- Setup check on the enable signal (EN) w.r.t. the clock.
- Hold check on the enable signal w.r.t. the clock.
- Minimum pulse width check on the gated clock.
- Some libraries also have internal checks (e.g., latch-based ICG with integrated latch).

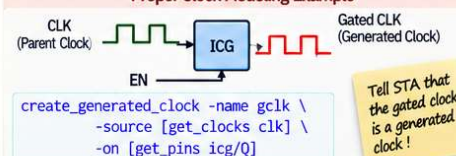
Clock Gating Cell (Latch-Based) and Timing Checks



5 Why must gated clocks be properly modeled?

- STA must understand that the gated clock is a real clock derived from the main clock.
- Proper clock modeling (e.g., create_generated_clock) ensures correct timing analysis of the sequential logic behind the gate.
- The tool can then perform clock gating checks, propagate uncertainties, and analyze the enable timing.
- Without proper modeling, the tool may treat the gated clock as a data signal, leading to incorrect analysis.

Proper Clock Modeling Example



6 What happens if a gated clock is not properly modeled?

- The tool may not recognize the gated clock as a clock.
- Sequential logic behind the gate may not be properly analyzed.
- Enable signal checks may be missing.
- Can lead to missed timing violations, functional failures, or silicon issues.
- May also prevent useful optimizations and power savings.

Impact of Missing Clock Gating Constraints

Unconstrained (Problem)	Properly Constrained
✗ Gated clock treated as data	✓ Gated clock recognized as clock
✗ Missing enable timing checks	✓ Enable setup/hold checks
✗ Sequential logic not timed	✓ Accurate timing analysis
✗ Possible functional failures	✓ Reliable power estimates
✗ Incorrect power/timing reports	✓ Fewer surprises in silicon
✗ Hard to debug	✓ Cleaner and safer design



Key Takeaway:

- Clock gating saves power, but only works correctly with proper constraints.
- Model gated clocks, check enable timing, and let STA analyze them like real clocks.

Constrain Clocks Wisely
Save Power Today, Performance Tomorrow

Learn the SDC TCL Language

SDC TCL Basics

Learn the SDC TCL Language

Same Tools
Used in EDA Tools!

Simple Syntax • Powerful Constraints • Better Designs!

= Write Be
= Control

1 Why is SDC TCL-based?

- SDC uses TCL (Tool Command Language) because it is a simple, flexible, and widely used scripting language in EDA tools.
- TCL allows easy customization, automation, and reuse of constraint scripts.
- The same TCL language is used in synthesis, place & route, and STA tools (e.g., Synopsys, Cadence, Siemens), so engineers can use one common scripting skill.
- It makes SDC easy to write, modify, and integrate into large flows.

**TCL gives flexibility. SDC gives meaning.
Together, they control your design timing!**

2 How does TCL syntax work in SDC?

- TCL syntax is simple and command-based.
- A command is followed by options and arguments (similar to a sentence).
- You can write one command per line or multiple commands in a script file (.sdc).
- TCL is case-sensitive and uses spaces to separate words.

Example TCL Syntax in SDC

```
create_clock -name clk -period 10 [get_ports clk]
```

Command Options (start with -) Argument (object list)

Command + Options + Arguments = Complete TCL statement

3 What are commands and options?

- Commands tell the tool what to do (e.g., create_clock, set_false_path, set_max_delay).
- Options (also called switches) start with a dash and modify the behavior of the command (e.g., -name, -from, -to).
- Some options take a value (e.g., -period 10), and some are flags (e.g., -verbose).
- Each command has its own set of valid options.

Example: Command and Options

```
set_false_path -from [get_ports A] -to [get_ports B]
```

Command Options



4 What are square brackets []?

- Square brackets are used for command substitution in TCL.
- The command inside [] is executed first, and its result is used as an argument to the outer command.
- In SDC, [] is commonly used to get design objects like ports, pins, cells, or clocks.

Example: Using [] to Get Objects

```
create_clock -period 10 [get_ports clk]
```

First, `get_ports clk` is executed and returns the port object list. Then it is used as an argument to `create_clock`.



Think of [] as "run this command and use its output".

5 What are braces {}?

- Braces {} are used to group text and treat it as a single argument.
- Everything inside {} is taken literally (no command substitution, no variable expansion).
- They are useful when the text contains spaces or special characters.

Example: Using {}

```
set_clock_groups -asynchronous \
  -group {clk1 clk2} \
  -group {clk3 clk4}
```

Braces keep the text together as one group!

Use {} when you want to pass a list or a string exactly as it is.

6 How do comments work in SDC?

- Comments in TCL (and SDC) start with the hash #.
- Everything after # on the same line is ignored by the tool.
- Comments are used to add notes, explanations, or temporarily disable a command.
- Good comments make your SDC file easy to read and maintain.

Example: Comments in SDC

```
# Create a 10 ns clock on port clk
create_clock -period 10 [get_ports clk]

# Set false path between two blocks
set_false_path -from [get_ports A] -to [get_ports B]

# set_max_delay 5 -from [get_pins U1/A] -to [get_pins U2/B]
# (This command is commented out)
```



Key Takeaway:

- SDC uses TCL, so learn basic TCL syntax to write clean and effective constraints.
- Use commands, options, [], {}, and comments correctly.
- A well-written SDC file is readable, reusable, and easy to maintain.

Good Constraints
Start with Good Comments

Same Design
Different Objects.
Different Constraints!

Understanding SDC Objects

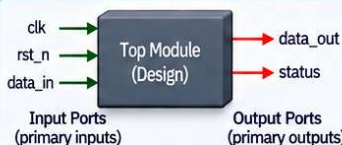
Target the Right Objects – Apply Correct Constraints – Get Accurate Results!

Identify Correct Objects
= Correct Timing
= Better Signoff

1 What are ports?

- Ports are the primary inputs and primary outputs of a design.
- They are the interface between the chip and the outside world.
- Input ports bring signals into the design, and output ports send signals out of the design.
- Used in constraints like `create_clock`, `set_input_delay`, `set_output_delay`, etc.

Example: Ports at the Top Level



Use `[get_ports]` to select ports in SDC!

2 What are pins?

- Pins are the input or output terminals of a cell (including sequential cells, combinational cells, and I/O cells).
- Each cell can have multiple pins (e.g., D, Q, CLK, A, Y).
- Pins are internal to the design (not just top-level).
- Used in constraints like `set_disable_timing`, `set_case_analysis`, `set_max_delay`, etc.

Example: Pins of a Flip-Flop

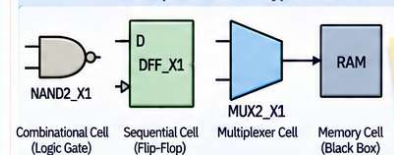


Use `[get_pins]` to select pins in SDC!

3 What are cells?

- Cells are the basic building blocks of a design (e.g., logic gates, flip-flops, multiplexers, I/O cells).
- Each cell contains pins and implements a specific function.
- Cells can be hierarchical (e.g., U1/U2) or from standard cells (e.g., NAND2_X1, DFF_X1).
- Used in constraints like `set_dont_touch`, `set_case_analysis`, `set_clock_gating_check`, etc.

Example: Different Types of Cells



4 What are clocks?

- Clocks are timing reference signals used for synchronous timing analysis.
- A clock defines the timing domain and is created in SDC using `create_clock`.
- Clocks can be defined on ports or pins, and can be propagated through the design.
- Clocks are used for setup/hold analysis, clock groups, uncertainty, etc.

Example: Defining a Clock on a Port

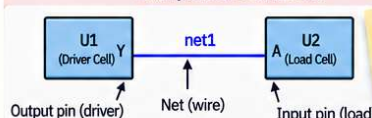
```
create_clock -name clk \
  -period 10 \
  [get_ports clk]
```

Clocks can be created on ports or pins!

5 What are nets?

- Nets are the interconnections (wires) between pins and cells in a design.
- A net carries a signal from a driver (output pin/port) to one or more loads (input pins).
- Nets are used to model signal connectivity and can be targeted in constraints (e.g., `set_max_delay`, `set_min_delay`).
- Nets can be internal or top-level.

Example: Net Connection



Use `[get_nets]` to select nets in SDC!

6 Why must SDC commands target correctly?

- Each SDC command is applied to specific design objects (ports, pins, cells, nets, or clocks).
- If the wrong object is targeted, the constraint may not apply to the intended logic.
- This can lead to incorrect timing analysis, missed violations, or over-constraining the design.
- Correct object targeting ensures accurate, meaningful, and reliable timing results.

Example: Correct vs. Incorrect Targeting

Correct (Targets the port) Incorrect (Targets the net)

```
set_input_delay 2.0 \
  [get_ports data_in]
```

Applies to the input port 'data_in'

```
set_input_delay 2.0 \
  [get_nets data_in]
```

No such port, constraint gives an error



Key Takeaway:

- Know the difference between ports, pins, cells, clocks, and nets.
- Use the correct SDC object for each constraint.
- Correct object targeting leads to accurate timing analysis and a successful signoff.

Right Objects
Right Constraints
Right Results

get_ports, get_pins, get_cells and get_clocks

Find the Right Objects – Apply the Right Constraints – Get Accurate Results !

Right Object
Right Command
Right Timing !

Use Wildcards
Use Commands
Avoid Errors
Get Better Results

1 What is get_ports?

- get_ports is used to get design ports (top-level inputs, outputs or bidirectional ports).
- Ports are the interface between the chip and the outside world.
- Used in commands like create_clock, set_input_delay, set_output_delay, etc.

Example: Top-level Ports



```
# Get a single port
get_ports clk
# Get a bus port
get_ports data_in[7:0]
```

Use get_ports to select top-level ports !

2 What is get_pins?

- get_pins is used to get pins of cells, including input pins, output pins and clock pins.
- Pins are internal to the design (not just top-level).
- Used for fine-grain constraints like set_false_path, set_max_delay, set_case_analysis, etc.

Example: Pins of a Cell



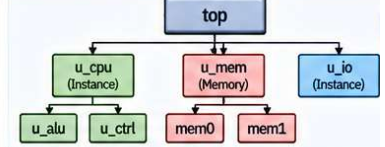
```
# Get a specific pin
get_pins U1/D
# Get all pins of a cell
get_pins U1/*
```

Use get_pins to select specific pins of a cell !

3 What is get_cells?

- get_cells is used to get design cells (instance hierarchical blocks).
- Cells can be standard cells, macros, memory hierarchical modules.
- Used for constraints like set_dont_touch, set_false_path, etc.

Example: Cells in a Design (Hierarchy)

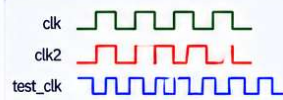


```
# Get a specific cell
get_cells u_cpu
# Get all cells in a hierarchy
get_cells top/*
```

4 What is get_clocks?

- get_clocks is used to get clock objects defined in SDC (using create_clock or propagated clocks).
- Clocks are timing reference signals.
- Used in commands like set_clock_groups, set_clock_latency, set_clock_uncertainty, etc.

Example: Defined Clocks



```
# Get a single clock
get_clocks clk
# Get multiple clocks using wildcard
get_clocks clk*
```

Use get_clocks to select clock objects defined in SDC !

5 How do wildcards work?

- Wildcards help you select multiple objects using patterns.
- * matches any number of characters.
- ? matches a single character.
- Useful when you have buses or repetitive naming.

Examples of Wildcards

Pattern	Matches	Example Usage
clk*	All names starting with clk	get_clocks clk*
data_*	All names starting with data_	get_ports data_*
U1/*	All pins inside U1	get_pins U1/*
mem?	mem0, mem1, mem2 ... (single character)	get_cells mem?
FF	All cells containing FF	get_cells *FF*

6 What happens if an object name is incorrect?

- If the object name is incorrect or does not exist return an empty list or give an error.
- Constraints may not be applied, leading to mismatches and incorrect results.
- Always check the output of get_* commands: reports to verify.

Example: Incorrect vs Correct Object Name

Incorrect (Object not found)	Correct
get_ports clk_1 Error: No objects matched 'clk_1'. (or returns an empty list)	get_ports clk # Returns: clk

Always check the output and fix the object name !

Key Takeaways:

- Use get_ports, get_pins, get_cells and get_clocks to select the correct design objects.
- Wildcards (*) and (?) help in selecting multiple objects.
- Always verify the object names to ensure constraints are applied correctly.

Know Your Objects
Constrain with Confidence

SDC Validation and Reports

First Checks After Receiving SDC – Make Sure Your Constraints Are Correct !

Check Early
Avoid Surprises
Save Time !

Right Constraints
= Reliable

1 Check file availability

Make sure the SDC file exists, is accessible, and is properly read without errors.



Useful Commands

```
# Check if file exists (Linux)
ls -lh constraints.sdc
# Read SDC in tool
read_sdc constraints.sdc
```

Sample Tool Output (Synopsys DC)

```
dc_shell> read_sdc constraints.sdc
Reading SDC file 'constraints.sdc' ...
Information: SDC file read successfully.
(CMD-013) Current design is 'mimo_top'.
(SDC-001) No errors found.
```

What to Look For

- File exists and is not empty
- No syntax errors while reading
- Correct file path

2 Check design name

Verify that the design name in the SDC matches the actual design loaded in the tool.



Useful Commands

```
# Check current design
current_design
# List all designs (if needed)
list_designs
```

Sample Tool Output

```
dc_shell> current_design
mimo_top
dc_shell> list_designs
dc_shell> list_designs
mimo_top
mimo_rx
mimo_tx
test_block
```

What to Look For

- Design name matches SDC
- No wrong or old design name
- Correct design is loaded

3 Check clocks

Verify that all expected clocks are created and properly recognized by the tool.



Useful Commands

```
# List all clocks
report_clocks
# If needed, check clock sources
report_clocks -verbose
```

Sample Tool Output

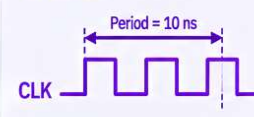
```
dc_shell> report_clocks
Clock Period Waveform Type
-----
clk 10.000 {0.000 5.000} primary
rx_clk 8.000 {0.000 4.000} primary
tx_clk 6.400 {0.000 3.200} primary
scan_clk 20.000 {0.000 10.000} primary
Total: 4 clocks
```

What to Look For

- All expected clocks are present
- No missing or extra clocks
- Clock names are as per design spec

4 Check clock periods

Verify that clock periods (and waveforms) are as intended in the SDC.



Useful Commands

```
# Report clock details
report_clocks
# Or check a specific clock
report_clocks clk
```

Sample Tool Output

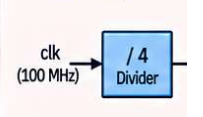
```
dc_shell> report_clocks clk
Clock name : clk
Type : primary
Period : 10.000
Waveform : {0.000 5.000}
Source : [get_ports clk]
Latency : 0.000
Uncertainty : 0.050
Transition : 0.100
```

What to Look For

- Periods match the specification
- Correct waveform (duty cycle)
- No unrealistic values
- Check uncertainties if set

5 Check generated clocks

Verify that all generated clocks (e.g., from dividers, PLLs) are correctly defined.



Useful Commands

```
# List generated clocks
report_clocks -generated
# Check clock tree
report_clock_tree
```

Sample Tool Output

```
dc_shell> report_clocks -generated
Clock Source Period
-----
gen_clk clk 40.000
div_clk clk 20.000
pll_clk pll_out 5.000
Total: 3 generated clocks
```

What to Look For

- All expected generated clocks are present
- Correct source and divider
- Periods match intended
- No missing generated clocks

Key Takeaways:

- Do these basic checks immediately after receiving an SDC file.

Validate Tool Output
for a Smooth Run

- Use tool reports to verify clocks, periods, and generated clocks.
- Catching issues early saves debug time and avoids wrong timing results.

Tapeout Tomorrow!

Validate Today
Better Timing
Tomorrow!

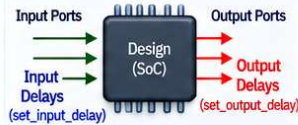
SDC Validation and Reports

Key Checks to Ensure Correct and Complete Constraints!

Check.
Report.
Fix.
Signoff with

1 Check I/O delays

Verify that input and output delays are defined for all primary I/O ports as per the interface specification.



Useful Commands

```
# List input delays
report_input_delay

# List output delays
report_output_delay
```

Sample Tool Output (Synopsys DC)

```
dc_shell> report_input_delay
Port      Clock  Delay  Max/Min
data_in   clk    2.000  max
addr[7:0] clk    1.500  max
rst_n     -      -      -

dc_shell> report_output_delay
Port      Clock  Delay  Max/Min
data_out  clk    3.000  max
valid     clk    2.500  max
Total: 2 output port(s)
```



What to Look For

- All primary input ports have input delays
- All primary output ports have output delays
- Delays match the interface specification
- No missing or unrealistic values

2 Check timing exceptions

Verify that all required timing exceptions (false paths, multicycle paths, max/min delay) are correctly defined.



Useful Commands

```
# Report false paths
report_exceptions -type false_path

# Report multicycle paths
report_exceptions -type multicycle_path

# Report max/min delay
report_exceptions -type max_min
```

Sample Tool Output (Synopsys DC)

```
dc_shell> report_exceptions -type false_path
From      To      Type
u_test/*  *      false_path
scan_en   *      false_path

dc_shell> report_exceptions -type multicycle_path
From      To      Setup  Hold
u_slow/*  u_fast/*  4      3

Total: 2 exceptions
```

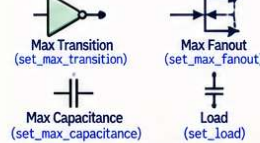


What to Look For

- All intended false paths are defined
- Multicycle paths have correct setup/hold values
- Any required max/min delay exceptions are present
- No unnecessary or overly broad exceptions

3 Check design rule constraints

Verify that important design rule constraints (DRC) like max transition, max capacitance, max fanout, and load constraints are specified.



Useful Commands

```
report_design_rules

# Or check individually
report_max_transition
report_max_capacitance
report_max_fanout
report_load
```

Sample Tool Output (Synopsys DC)

```
dc_shell> report_design_rules
Rule      Limit  Violations
max_transition  0.200  0
max_capacitance 0.500  0
max_fanout     10     0
max_load       0.200  0

Total violations: 0
```



What to Look For

- Required DRC constraints are defined
- No violations (or only expected ones)
- Values are per design technology/spec
- Constraints applied to correct objects

4 Check uncertainty and latency

Verify clock uncertainty and latency values are defined and reasonable for all clocks.



Useful Commands

```
report_clock_uncertainty
report_clock_latency
report_clocks
```

Sample Tool Output (Synopsys DC)

```
dc_shell> report_clock_uncertainty
Clock      Setup  Hold
clk        0.050 0.050

dc_shell> report_clock_latency
Clock      Source Latency  Network Latency
clk        0.100 0.200
```

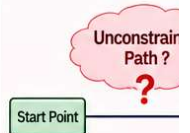


What to Look For

- Uncertainty defined for all clocks
- Source and network latency are specified
- Values are realistic (not zero unless intended)
- Consistent with clock tree and methodology

5 Check unconstrained paths

Verify that there are no paths in the design. All be covered by clocks or



Useful Commands

```
report_timing -unconstrained
report_analysis_coverage
# (Tool specific)
check_timing -unconstrained
```

Sample Tool Output (Synopsys DC)

```
dc_shell> report_timing -unconstrained
Warning: Found 5 unconstrained paths.
Startpoint      Endpoint
u1/q            u2/q
u3/q            u4/q
u5/q            u6/q
.... (2 more)
Total: 5 unconstrained paths
```



What to Look For

- No (or minimal) unconstrained paths
- Investigate and constrain
- Check both setup and hold
- Use analysis coverage report



Key Takeaways:

- Always validate SDC after receiving or modifying it.
- Use tool reports to check I/O delays, exceptions, DRC, uncertainty/latency, and unconstrained paths.
- A clean report gives confidence in your timing signoff and avoids late surprises.

Good Constraints
Lead to Good Results

Check Today
Fix Tomorrow
Save Iterations!

check_timing

Find Constraint Issues Early – For a Clean and Reliable Signoff!

A Simple Check
A Big Impact
Run check_timing

1 What is check_timing?

- check_timing is a built-in STA command (e.g., in Synopsys PrimeTime) that checks the completeness and consistency of your timing constraints (SDC).
- It analyzes the design and reports potential constraint issues.
- It does not perform full timing analysis; it only checks for missing or incorrect constraints.

Example Command

```
pt_shell> check_timing
Checking timing constraints ...
```

Quick health check for your SDC!

2 What problems can it detect?

It can detect many common constraint issues, such as:

- Unconstrained paths
- Missing clocks
- Missing I/O delays
- Missing required or generated clocks
- Incorrect or incomplete clock definitions
- Overly broad or conflicting constraints
- Objects without constraints (ports, pins, etc.)



Helps you catch issues before running full timing analysis or tapeout signoff.

3 How does it find unconstrained paths?

- check_timing traces all timing paths in the design.
- It checks whether each path has a valid clock required timing constraints (setup/hold).
- If a path is not covered by any clock (or is cut off), it is reported as an unconstrained path.

Example Tool Output

```
pt_shell> check_timing
Warning: Found 5 unconstrained paths.
Startpoint      Endpoint
u1/q            u2/d
u3/q            u4/d
u5/q            u6/d
u_mem/q         u_ctrl/d
u_core/out      u_top/in
```

4 How does it detect missing clocks?

- It checks if all sequential elements (flip-flops, latches) are driven by a defined clock.
- If it finds sequential elements that are not associated with any clock (no create_clock or generated clock), it reports them as missing clocks.

Example Tool Output

```
pt_shell> check_timing
Warning: Sequential elements without clocks:
Instance      Type
u_ff1         DFF
u_ff2         DFF
u_reg_file/u_ff3 DFF
```



5 How does it detect missing I/O constraints?

- It checks if all primary input ports have input delays (set_input_delay) and all primary output ports have output delays (set_output_delay).
- If any input or output port is missing these constraints, it reports them.

Example Tool Output

```
pt_shell> check_timing
Warning: Primary input ports without input delay:
Port
data_in
addr[7:0]
mode
Warning: Primary output ports without output delay:
Port
data_out
valid
```



6 Why should PD engineers run it?

- Catches major constraint issues early.
- Saves time by avoiding unnecessary full STA runs.
- Helps ensure complete and correct SDC before timing closure.
- Reduces the risk of missing paths, which can cause silicon failures.
- It is a quick and easy sanity check and should be part of the standard flow.

Run check_timing first, so you can focus on real timing closure instead of basic constraint errors!

**Key Takeaways:**

- check_timing checks the completeness and consistency of your SDC.
- It reports unconstrained paths, missing clocks, missing I/O delays, and other constraint issues.
- Always run it first to ensure a clean and reliable timing environment.

Clean Constr
Build Reliable

Reports Give
Visibility
Build a Reliable Design!

Important SDC Reports

Use the Right Reports – Verify Your Constraints – Ensure Clean Signoff!

Check!
Understand
Fix Issues!
Signoff with

1 What does report_clocks show?

- Shows all defined clocks in the design.
- Displays clock name, period, waveform, source, latency, and other clock properties.
- Helps verify that all expected clocks are created correctly.



Check if all
clocks are
present and
correct!

Example Command
report_clocks

Sample Tool Output (Synopsys DC)

```
dc_shell> report_clocks
```

Clock Name	Period	Waveform	Type	Latency	Source
clk	10.000	{0.000 5.000}	primary	0.000	[get_ports clk]
rx_clk	8.000	{0.000 4.000}	primary	0.000	[get_ports rx_clk]
tx_clk	12.000	{0.000 6.000}	primary	0.000	[get_ports tx_clk]
scan_clk	20.000	{0.000 10.000}	primary	0.000	[get_ports scan_clk]

2 What does report_clock_domains show?

- Shows different clock domains in the design.
- Lists which clocks belong to each domain.
- Helps identify CDC (Clock Domain Crossing) scenarios.
- Useful to verify multi-clock designs.



Example Command
report_clock_domains

Sample Tool Output (Synopsys DC)

```
dc_shell> report_clock_domains
```

Clock Domain	Clocks	Type
domain_0	clk	exclusive
domain_1	rx_clk	exclusive
domain_2	tx_clk	exclusive
domain_3	scan_clk	exclusive

3 What does report_timing show?

- Shows detailed timing paths (startpoint to endpoint)
- Displays delays, arrival/required times, slack, and violations.
- Helps verify that timing constraints are working as



Example Command
report_timing

Sample Tool Output (Synopsys DC)

```
dc_shell> report_timing
```

Startpoint: u_ff1/Q (rising edge-triggered flip-flop clocked)
Endpoint : u_ff2/D (rising edge-triggered flip-flop clocked)
Path Group: clk

Point	Incr	Path
clock clk (rise edge)	0.000	0.000
u_ff1/Q (reg)	0.120	0.120
u_comb1/U1 (cell)	1.250	1.370
u_ff2/D (reg)	0.000	1.450
slack (MET)		0.550

4 How do you report unconstrained paths?

- Use report_timing with -unconstrained option.
- Shows paths that are not covered by any clock or I/O constraints.
- Important to ensure there are no missing constraints.

Example Command
report_timing -unconstrained

Sample Tool Output (Synopsys DC)

```
dc_shell> report_timing -unconstrained
```

Warning: Found 3 unconstrained paths.

Startpoint	Endpoint
u_async_reg/Q	u_sync_reg/D
u_ctrl/a	u_out1/Z
u_mem/data_out[3]	u_top/out[7]

Total: 3 unconstrained paths



Ideally,
this report
should be empty!

5 How do you check timing exceptions?

- Use report_exceptions to view all defined exceptions (false path, multicycle path, etc.).
- Helps verify that exceptions are correctly applied.
- Ensures no unintended or overly broad exceptions.

Example Command
report_exceptions

Sample Tool Output (Synopsys DC)

```
dc_shell> report_exceptions
```

From	To	Type	Value
u_test1/Q	scan_en	false_path	-
u_ctrl1/a	u_mem/*	multicycle	2
u_async/*	u_sync/*	false_path	-
clk1	clk2	max_delay	5.000

Total: 3 exceptions



Verify all
exceptions are
intended!

6 What reports should a PD engineer

Key reports to review for SDC validation and signoff:

- ✓ report_clocks – Verify all clocks
- ✓ report_clock_domains – Check clock domains (CDC)
- ✓ report_timing – Check timing and slack
- ✓ report_timing -unconstrained – Ensure no unconstrained paths
- ✓ report_exceptions – Verify timing exceptions
- ✓ report_design_rules – Check DRC constraints (max transition, max capacitance, etc.)
- ✓ check_timing – Overall SDC consistency check



Review all key reports to ensure con
accurate, and clean constraints for relia

**Key Takeaways:**

- SDC reports give visibility into clocks, timing, exceptions, and constraint coverage.
- Always verify reports after making changes to SDC.
- A clean set of reports leads to a correct design, easier debug, and successful signoff.

Good Rep
Build Great

Small Mistakes
Big Timing Issues!

Common SDC Errors

Identify · Understand · Fix · Get a Clean Signoff!

Check Comm
Check Objec
Check Repor
Avoid SDC

1 Missing create_clock

No clock is created for a primary clock port.

Missing clock definition
X # create_clock -name clk -period 10 [get_ports clk]

Tool Output (Synopsys DC)

```
dc_shell> check_timing
```

Warning: No clocks defined in the design.
All paths are unconstrained.
(TIM-001)



Fix

Create the clock with correct period.

create_clock -name clk -period 10 [get_ports clk]

2 Missing generated clock

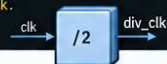
Generated clocks (e.g., from dividers/PLLs) are not defined.

Missing generated clock
X # create_generated_clock ...

Tool Output (Synopsys DC)

```
dc_shell> check_timing
```

Warning: Found sequential element 'u_div/Q' with no generated clock.
(TIM-004)



Fix

Define the generated clock.

create_generated_clock -name div_clk \

-source [get_ports clk] -divide_by 2 \

[get_pins u_div/Q]

3 Wrong clock period

Clock period is set incorrectly (too high or too low).

Wrong period (actual is 10ns, given 20ns)
X create_clock -name clk -period 20 [get_ports clk]

Tool Output (Synopsys DC)

```
dc_shell> report_clocks
```

Clock	Period	Waveform
clk	20.000	{0.000 10.000}

Results in incorrect timing analysis
(may hide real violations or create false ones).



Fix

Use the correct period.

create_clock -name clk -period 10 [get_ports clk]

4 Wrong clock source

Generated clock is defined with wrong s

Wrong source (using output instead of input)
X create_generated_clock -name gen_clk \

-source [get_pins u_div/Q] -divide_by 2 [get_ports clk]

Tool Output (Synopsys DC)

```
dc_shell> check_timing
```

Warning: Cannot find valid clock source for generated clock 'gen_clk'.
(TIM-053)



Fix

Use the correct source pin/port.

create_generated_clock -name gen_clk \

-source [get_ports clk] -divide_by 2 [get_ports clk]

5 Undefined clock name

Using a clock name that is not defined.

Clock name 'myclk' is not defined
X set_clock_uncertainty 0.1 [get_clocks myclk]

Tool Output (Synopsys DC)

```
dc_shell> set_clock_uncertainty 0.1 [get_clocks myclk]
```

Error: No such clock 'myclk'.
(CMD-012)



Fix

Use a valid clock name (check with report_clocks).

report_clocks

6 Missing I/O delays

Input or output delays are not specified.

No input/output delays
X # set_input_delay ...
X # set_output_delay ...

Tool Output (Synopsys DC)

```
dc_shell> check_timing
```

Warning: Primary input port 'data_in' has no input delay.
Warning: Primary output port 'data_out' has no output delay.
(TIM-010)



Fix

Add input and output delays.

set_input_delay 2.0 -clock clk [get_ports data_in]

set_output_delay 2.0 -clock clk [get_ports data_out]

7 Missing exceptions

Required false/multicycle paths are not defined.

No false path for test logic
X # set_false_path ...
X # No multicycle path
X # set_multicycle_path ...

Tool Output (Synopsys DC)

```
dc_shell> report_exceptions
```

Information: No timing exceptions defined in the current design.
(TIM-061)



Fix

Add the required exceptions.

set_false_path -from [get_clocks test_clk] \

-to [all_registers]

8 Wrong object name

Using incorrect port/pin/cell name in SDC

Typo in port name (data_in) instead of data_in1
X set_input_delay 2.0 -clock clk [get_ports data_in1]

Tool Output (Synopsys DC)

```
dc_shell> set_input_delay 2.0 -clock clk [get_ports data_in1]
```

Error: No objects matched 'data_in1'.
(CMD-007)
Error: Stack is empty.



Fix

Use the correct object name (verify port/pin/cell name).

get_ports data_in* # check available

set_input_delay 2.0 -clock clk [get_ports data_in]

```
set_clock_uncertainty 0.1 [get_clocks clk]
```

```
set_output_delay 2.0 -clock clk [get_ports data_out]
```

```
set_multicycle_path 2 -from A -to B
```

Key Takeaways:

- Most SDC errors come from missing, incorrect, or inconsistent constraints.
- Always check tool reports (check_timing, report_clocks, report_exceptions, etc.).
- Verify object names and clock definitions.
- Fixing SDC errors early saves debug time and ensures reliable timing signoff.

*Correct Const
Stronger Des*

No Unconstrained Paths = Reliable Signoff!

Unconstrained Paths Debugging

Find • Understand • Fix • Ensure Complete Timing Coverage!

Constrain for a Better Tomorrow

1 What is an unconstrained path?

- An unconstrained path is a timing path for which the tool cannot determine proper timing constraints (startpoint clock, endpoint clock, or I/O delays).
- It is not analyzed for setup/hold, so real timing violations may be missed.
- Having unconstrained paths can lead to silicon failures.

2 Why do paths become unconstrained?

- Missing clocks (primary or generated).
- Missing I/O delays.
- Incorrect or incomplete clock definitions.
- Missing exceptions (for intentional cases).
- Wrong object names in constraints.
- Logic not connected to any clock (e.g., test logic, debug logic).

Common Causes

No clock defined

Missing generated clock

No I/O delays

Wrong/missing exceptions

Incorrect object names

3 How do missing clocks cause this?

- If a flip-flop or register is not associated with the tool cannot determine the launch or capture clock.
- All paths starting from or ending at such element become unconstrained.

Example: check_timing report (Synopsys DC)

```
dc_shell> check_timing
Warning: Found 12 unconstrained paths.
Startpoint      Endpoint
-----
u_reg1/Q        u_logic1/D
u_reg2/Q        u_reg3/D
u_debug_reg/Q   u_out_reg/D
u_test_reg/Q    u_top/u_cnt_reg/D
u_fifo_reg/Q    u_ctrl_reg/D
Total: 12 unconstrained paths
```

4 How do missing generated clocks cause this?

- Blocks like dividers, PLLs, muxes, or clock gating create new clocks (generated clocks).
- If generated clocks are not defined using create_generated_clock, the downstream registers are not associated with any clock.
- All paths in that clock domain become unconstrained.

Example: Divider without generated clock

Correct Constraint:

```
create_generated_clock -name div_clk \
-source [get_ports clk] -divide_by 4 [get_pins u_div/U/Q]
```

5 How do you find unconstrained endpoints?

- Use report_timing with -unconstrained option.
- Check startpoints and endpoints.
- You can also use report_analysis_coverage to see overall constraint coverage.

Example: report_timing -unconstrained (Synopsys DC)

```
dc_shell> report_timing -unconstrained
Warning: Found 12 unconstrained paths.
Startpoint      Endpoint
-----
u_reg1/Q        u_logic1/D
u_reg2/Q        u_reg3/D
u_debug_reg/Q   u_out_reg/D
u_test_reg/Q    u_top/u_cnt_reg/D
u_fifo_reg/Q    u_ctrl_reg/D
Total: 12 unconstrained paths
```

List of unconstrained endpoints!

Example: report_analysis_coverage

```
dc_shell> report_analysis_coverage
Design : top
Category  Total  Constrained  Unconstrained  Coverage
Endpoints 1250   1200         50              96.8%
```

Check coverage!

6 How do you fix unconstrained paths?

- Create missing primary clocks (create_clock).
- Define generated clocks (create_generated_clock).
- Add input/output delays (set_input_delay / set_output_delay).
- Apply required timing exceptions (set_false_path, set_multicycle_path).
- Verify correct object names (use get_ports, get_pins).
- Re-run check_timing and ensure 0 unconstrained paths.

Example Fix Flow

```
dc_shell> create_clock -name clk -period 10 [get_ports clk]
dc_shell> create_generated_clock -name div_clk -source [get_ports clk] \
-divide_by 4 [get_pins u_div/U/Q]
dc_shell> set_input_delay 2.0 -clock clk [get_ports data_in]
dc_shell> set_output_delay 2.0 -clock clk [get_ports data_out]
dc_shell> report_timing -unconstrained
Info: No unconstrained paths found.
```

Key Takeaways:

- Unconstrained paths hide real timing issues and can lead to silicon failures.
- Always define all clocks (primary and generated), I/O delays, and exceptions.
- Use reports like check_timing and report_analysis_coverage to find and fix unconstrained paths.
- Ensure 0 unconstrained paths before signoff.

*Full Cons
Complete
Reliable*

**Right Constraints
Right Timing
Successful Tapeout!**

SDC Signoff Checklist

Review • Verify • Fix • Ensure Clean Signoff!

CLEAN SDC = RELIABLE CHIP!

**Check Early
Check Comp
Signoff with**

1 Are all clocks defined?

- Check that all primary clocks are defined.
- Verify clock names, sources and periods.
- Ensure no missing clocks.

Useful Command(s)

```
dc_shell> report_clocks
```

Clock	Period	Waveform	Source
clk	10.000	{0.000 5.000}	port
rx_clk	8.000	{0.000 4.000}	port

2 Are all generated clocks defined?

- Check generated clocks (dividers, PLLs, muxes, gated clocks).
- Verify source, division and waveform.
- Ensure no missing generated clocks.

Useful Command(s)

```
dc_shell> report_clocks -generated
```

Clock	Source	Divide_by
clk	port	1
clk_div2	clk	2
clk_div4	clk	4

3 Are clocks propagated after CTS?

- Ensure clocks are propagated (after CTS).
- Check for ideal/propagated status.
- No clocks should remain ideal at signoff.

Useful Command(s)

```
dc_shell> report_clocks
```

Clock	Propagated
clk	true
rx_clk	true
scan_clk	true

4 Is uncertainty updated?

- Check setup and hold uncertainties.
- Use realistic values (post-CTS, post-route).
- Ensure values match signoff methodology.

Useful Command(s)

```
dc_shell> report_clock_uncertainty
```

Clock	Setup	Hold
clk	0.050	0.050
rx_clk	0.080	0.080
scan_clk	0.100	0.100

5 Is latency constrained?

- Check source and network latency.
- Verify values (especially post-route).
- Ensure latency is not unconstrained.

Useful Command(s)

```
dc_shell> report_clock_latency
```

Clock	Source Latency	Network Latency
clk	0.100	0.100
rx_clk	0.150	0.150

6 Are all endpoints constrained?

- No unconstrained paths/endpoints.
- Check using reports.
- All primary inputs, outputs and registers should be constrained.

Useful Command(s)

```
dc_shell> check_timing
Warning: 0 unconstrained paths.
```

7 Are CDC exceptions correct?

- Verify asynchronous clock groups.
- Check set_clock_groups / set_false_path.
- Ensure no missing or incorrect CDC exceptions.

Useful Command(s)

```
dc_shell> report_clock_groups
dc_shell> report_exceptions -type clock_group
```

8 Are scan constraints correct?

- Check scan clocks, scan enables and scan mode.
- Verify false paths for scan.
- Ensure no functional paths are accidentally cut.

Useful Command(s)

```
dc_shell> report_clocks | grep scan
```

Clock	Period	Waveform
scan_clk	20.000	{0.000 10.000}

9 Are I/O delays correct?

- Check input and output delays on all ports.
- Verify min and max values.
- Ensure they match interface specifications.

Useful Command(s)

```
dc_shell> report_input_delay
```

Port	Min	Max	Clock
data_in	0.500	2.000	clk

10 Are OCV and sign constraints applied?

- Check OCV/AOCV/POCV.
- Verify derates, PVT corner views.
- Ensure signoff constraint as per methodology.

Useful Command(s)

```
dc_shell> report_analysis_
```

```
dc_shell> report_timing -unconstrained
Info: No unconstrained paths found.
```

```
Group 1 : clk_a
Group 2 : clk_b
Type    : asynchronous
```

```
dc_shell> report_exceptions | grep scan
set_false_path -from [get_ports scan_in] \
               -to [get_ports scan_out]
```

```
dc_shell> report_output_delay
Port      Min    Max    Clock
data_out  0.500  2.000  clk
```

```
dc_shell> report_timing
dc_shell> report_ocv
dc_shell> report_library
```

**Key Takeaways:**

- ✓ A clean and complete SDC is essential for accurate timing analysis.
- ✓ Always review key reports and fix all warnings before signoff.
- ✓ Follow a consistent methodology and cross-check with design specifications.
- ✓ SDC signoff ensures a reliable, high-performance and manufacturable chip.

**Complete Checklist = Clean SDC = Reliable Silicon !**

"Good Constraints Build Great Chips !"

Verify To
for a Better TConstraints
Turn RTL into Real Silicon !**Complete SDC File Example**

A Realistic and Practical SDC for Timing Signoff

Same Command
Real Designs**1 create_clock**

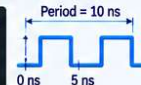
Defines a primary clock on a port.

```
# Create a 100 MHz clock (10 ns period)
create_clock -name clk \
  -period 10.0 [get_ports clk]
```

2 Clock waveform

Defines the clock waveform (rise and fall times).

```
# 50% duty cycle (rise at 0, fall at 5ns)
create_clock -name clk \
  -period 10.0 -waveform {0.0 5.0} \
  [get_ports clk]
```

**3 Generated clock**

Defines a clock generated from a source clock (e.g., divider).

```
# Create a generated clock (divide by 2)
create_generated_clock -name clk_div2 \
  -source [get_ports clk] -divide_by 2 \
  [get_pins u_divider/clk_out]
```

4 Virtual clock

Defines a virtual clock (no physical port, used for I/O timing).

```
create_clock -name virt_clk -period 10.0
# No port is specified (virtual clock)
```

Complete SDC File (Example)

```
1 # =====
2 # Example SDC File
3 # Design : simple_soc
4 # Purpose: Timing constraints for synthesis, PnR and signoff
5 # =====
6
7 # 1. Create primary clock with waveform
8 create_clock -name clk -period 10.0 -waveform {0.0 5.0} [get_ports
9
10 # 2. Generated clock (divide by 2)
11 create_generated_clock -name clk_div2 -source [get_ports clk] \
12   -divide_by 2 [get_pins u_divider/clk_out]
13
14 # 3. Virtual clock for I/O
15 create_clock -name virt_clk -period 10.0
16
17 # 4. Input delay
18 set_input_delay 2.0 -clock clk [get_ports {data_in reset_n}]
19
20 # 5. Output delay
21 set_output_delay 2.0 -clock clk [get_ports {data_out}]
22
23 # 6. Clock uncertainty
24 set_clock_uncertainty 0.1 [get_clocks clk]
25
26 # 7. Clock latency
27 set_clock_latency -source 0.2 [get_clocks clk]
28 set_clock_latency -network 1.0 [get_clocks clk]
29
30 # 8. False path (ignore reset paths)
31 set_false_path -from [get_ports reset_n] -to [all_registers]
32
33 # 9. Multicycle path
34 set_multicycle_path 2 -from [get_ports start] -to [get_registers u_ct
35 set_multicycle_path 1 -from [get_ports start] -to [get_registers u_ct
36
37 # =====
38 # End of SDC
39 # =====
```

5 Input delay

Specifies input delay relative to a clock (for external devices).

```
# Input arrives 2 ns after clock edge
set_input_delay 2.0 -clock clk \
  [get_ports {data_in reset_n}]
```

6 Output delay

Specifies output delay relative to a clock.

```
# Output required 2 ns before next clock edge
set_output_delay 2.0 -clock clk \
  [get_ports {data_out}]
```

7 Clock uncertainty

Specifies clock uncertainty (jitter, skew, etc.).

```
# 0.1 ns uncertainty on clock
set_clock_uncertainty 0.1 [get_clocks clk]
```

8 Clock latency

Specifies source and/or network latency.

```
# 0.2 ns source latency
set_clock_latency -source 0.2 [get_clocks clk]
# 1.0 ns network latency
set_clock_latency -network 1.0 [get_clocks clk]
```

9 False path

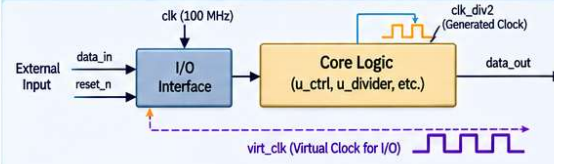
Specifies paths that should be ignored in timing analysis.

```
# Ignore paths from reset to all registers
set_false_path -from [get_ports reset_n] \
  -to [all_registers]
```

10 Multicycle path

Allows more than one clock cycle for a path.

```
# Give 2 cycles for a path
set_multicycle_path 2 -from [get_ports start] \
  -to [get_registers u_ctrl/*]
set_multicycle_path 1 -from [get_ports start] \
  -to [get_registers u_ctrl/*] -end
```

Design Context (Example Block Diagram)**Key Takeaways:**

- This SDC covers the most common and essential constraints used in real projects.
- Always verify constraints with reports (report_clocks, report_timing, check_timing, etc.).
- Modify values (period, delays, uncertainty, etc.) as per your design and technology.

Good Constraints
Lead to Great SMore Constraints
Better Timing !**Important SDC Constraints (Advanced but Essential)**

Complete Your Timing Constraints • Better Accuracy • Reliable Signoff

Constrain Acc
Design Confic**1 Clock groups**

- Used to define asynchronous clock domains.
- Prevents false timing analysis between unrelated clocks.
- Commonly used with -asynchronous option.

Example Command

```
# Define asynchronous clock groups
set_clock_groups -asynchronous \
  -group {clk_a clk_b} \
  -group {clk_c clk_d}
```

Concept Diagram**2 Case analysis**

- Used to set a constant value on a signal (e.g., mode select, test enable).
- Helps in analyzing a specific mode of operation.
- Reduces false paths and unrealistic timing.

Example Command

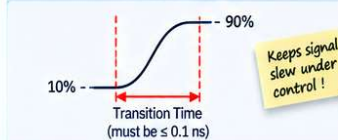
```
# Assume test_mode is 0
set_case_analysis 0 [get_ports test_mode]
```

Concept Diagram**3 Max transition**

- Specifies the maximum allowed transition time on a net.
- Helps avoid slow slews which can cause reliability and timing issues.

Example Command

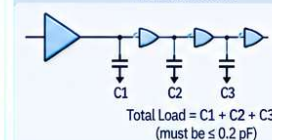
```
# Set max transition to 0.1 ns on all outputs
set_max_transition 0.1 [get_ports *]
```

Transition Time**4 Max capacitance**

- Specifies the maximum load capacitance allowed on a net.
- Helps to control excessive loading.
- Prevents large RC delays and degrades signal integrity.

Example Command

```
# Set max capacitance to 0.2 pF on
set_max_capacitance 0.2 [get_ports *]
```

Net Capacitance**5 Max fanout**

- Specifies the maximum number of loads that a net can drive.
- Helps to control buffering and maintain timing and signal quality.

Example Command

```
# Set max fanout to 10 on all outputs
set_max_fanout 10 [get_ports *]
```

Fanout Example**6 Output load**

- Specifies the expected external load capacitance on an output port.
- Used to model the real environment (e.g., PCB load, next stage input).

Example Command

```
# Set output load to 0.5 pF on all outputs
set_load 0.5 [get_ports data_out *]
```

Output Load Model**7 Driving cell**

- Specifies the reference driving cell for an input port.
- Used to model the input transition based on a real library cell.
- Helps get realistic input slew and timing.

Example Command

```
# Use INX4 as driving cell for all inputs
set_driving_cell -lib_cell INX4 [get_ports *]
```

Driving Cell Model**8 Propagated clock**

- Specifies that the clock is propagated the clock network (after CTS).
- Used for post-CTS timing analysis.
- Ensures real clock latency and skew are

Example Command

```
# Set clock as propagated
set_propagated_clock [get_clocks *]
```

Before vs After

Ideal Clock (Pre-CTS) | Propagated C

Key Takeaways:

- These constraints help create a realistic and complete timing environment.
- Use them along with basic constraints (clocks, I/O delays, exceptions, etc.).
- Properly constrained designs lead to better timing accuracy and reliable signoff.

✓

Constrain Smart → Analyze Accurate → Signoff Confident

1/5 Introduction & Project Setup

Understand the flow, tools and create your working environment

1. SDC and Cadence Flow

- SDC (Synopsys Design Constraints) is a text file with timing constraints.
- In Cadence flow it is used in:
 - Genus → Synthesis
 - Innovus → Physical Design (MMMC, CTS, etc.)
 - Tempus → Signoff STA
- You create/edit the SDC file, load it into the tool, validate it and update it through the flow.

SDC is usually a TCL text file, not a GUI button!

2. Project Directory Structure (Example)

```

MIMO_Project/
├── rtl/
│   ├── top.v
│   └── modules.v
├── lib/
│   ├── stdcell.lib
│   ├── lef/
│   │   ├── tech.lef
│   │   └── stdcell.lef
│   └── constraints/
│       └── top.sdc
├── scripts/
│   └── genus.tcl
└── reports/
          
```

2. Create Working Directory

```

$ mkdir MIMO_Project
$ cd MIMO_Project
$ mkdir constraints
$ touch constraints/top.sdc
$ ls constraints/
top.sdc
$ vim constraints/top.sdc # or gedit
          
```

Now you can start writing your SDC using a text editor.

2/5 Write the SDC File – Basic Constraints

Start with clocks, I/O delays and essential constraints

1. Primary Clock

Define the main clock with period and waveform.

```

# Primary Clock
create_clock -name CLK \
-period 10.0 \
-waveform {0 5} \
[get_ports clk]
          
```

2. Clock Uncertainty

Model clock jitter, skew, etc.

```

set_clock_uncertainty 0.1 \
[get_clocks CLK]
          
```

3. Input Delay

Specify delay for external

```

set_input_delay -max 2.0 \
[get_ports data_in] \
[get_ports data_in]
          
```

4. Output Delay

Specify delay for output signals.

```

set_output_delay -max 2.0 -clock CLK \
[get_ports data_out] \
set_output_delay -min 0.5 -clock CLK \
[get_ports data_out]
          
```

5. Reset/Exception

Ignore asynchronous reset

```

set_false_path -from [get_ports reset]
          
```

Key Point: Start with clocks and I/O delays. These are the most important constraints.

3/5 Add Design Rule and Interface Constraints

Guide the tool with realistic design rules and environment information

1. Driving Cell (Input Transition)

Specify the reference driving cell for inputs.

```

set_driving_cell -lib_cell BUF4 \
[get_ports data_in]
          
```

2. Output Load

Specify the expected external load.

```

set_load 0.05 [get_ports data_out]
          
```

3. Design Rule Constraints

Set limits for better implementation.

```

set_max_transition 0.2 [get_ports *]
set_max_capacitance 0.1 [get_ports *]
set_max_fanout 10 [get_ports *]
          
```

These constraints help control:

- Transition/Slew
- Capacitive loading
- Fanout count
- Signal integrity and timing quality

4/5 Advanced Constraints

Handle generated clocks, clock groups, case analysis, multicycle paths

1. Generated Clock

Define clocks created from a source clock (e.g., divider, PLL).

```

create_generated_clock -name CLK_DIV2 \
-source [get_ports clk] \
-divide_by 2 \
[get_pins u_div/clk_out]
          
```

2. Virtual Clock

Define a virtual clock (no physical port).

```

create_clock -name VIRT_CLK \
-period 10.0
          
```

Used for I/O timing (e.g., external interface) when no physical clock exists in the design.

3. Clock Groups

Declare asynchronous

```

set_clock_group \
-group [CLK] \
-group [CLK_DIV2]
          
```

4. Case Analysis

Set constant value for signals (e.g., test mode).

```

set_case_analysis 0 [get_ports test_mode]
          
```

5. Multicycle Path

Allow more than one cycle for a path.

```

set_multicycle_path 2 -setup -from [get_ports start] \
-to [get_registers u_ctrl/*]
          
```

6. Additional Exceptions

Example: false path

```

set_clock_tree -from [get_ports clk] \
-to [get_ports data_in]
          
```

5/5 Load, Verify and Use the SDC in Cadence Tools

Read the SDC, check reports and run the flow (Genus → Innovus)

1. Run in Genus (Synthesis)

Start genus and read RTL + SDC.

```

$ genus
(genus)> read_hdl ../rtl/top.v
(genus)> elaborate top
(genus)> read_sdc ../constraints/top.sdc
(genus)> check_timing
(genus)> report_clocks
(genus)> report_timing
(genus)> syn_generic
(genus)> syn_rpe
(genus)> write_hdl ../outputs/top_netlist.v
(genus)> write_sdc ../outputs/top.sdc
          
```

Important Reports to Check:

- report_clocks (all clocks defined)
- check_timing (unconstrained paths, missing I/O, etc.)
- report_timing (verify timing)
- report_exceptions
- report_design_rules

2. Use SDC in Innovus (Physical Design)

Create constraint mode and set analysis view (simplified).

```

innovus> create_constraint_mode -name FUNC \
-sdc_files ../constraints/top.sdc
innovus> create_library_set -name LIB_SS \
-timing ../lib/slow.lib
innovus> create_rc_corner -name RC_TYP -qz tech_file ../tech/typ.tch
innovus> create_delay_corner -name DELAY_SS \
-library_set LIB_SS -rc_corner RC_TYP
innovus> create_analysis_view -name FUNC_SETUP \
-constraint_mode FUNC -delay_corner DELAY_SS
innovus> set_analysis_view -setup {FUNC_SETUP} -hold {FUNC_SETUP}
          
```

3. After CTS – Propagate Clocks

Once clock tree is built, use propagated clocks.

```

innovus> set_propagated_clock [all_clocks]
          
```

Then verify:

- report_clocks (latency, skew, propagated status)
- report_timing (post-CTS)
- check_timing (no unconstrained paths)

Final Goal:
A clean, complete and accurate SDC for reliable timing

2/5 Write the SDC File – Basic Constraints

Start with clocks, clock waveform, generated clocks, virtual clocks, I/O delays and uncertainty

1 Primary Clock (create_clock)

Defines the main clock on a port or pin with period and waveform.

```

create_clock -name CLK \
-period 10.0 \
-waveform {0 5} \
[get_ports clk]
          
```

What it means?

- Clock name: CLK
- Period: 10 ns (100 MHz)
- Waveform: rising at 0 ns, falling at 5 ns (50% duty cycle)
- Applied on port: clk

Defines the clock for timing analysis!

2 Clock Uncertainty (set_clock_uncertainty)

Models clock jitter, skew and other uncertainties.

```

set_clock_uncertainty 0.1 [get_clocks CLK]
          
```

Why?

- Accounts for clock jitter, skew, and modeling inaccuracies.
- Reduces the effective timing budget, making the design robust.

Tool subtracts uncertainty from timing budget!

3 Input Delay (set_input_delay)

Specifies input arrival time relative to a clock (for external signal)

```

set_input_delay -max 2.0 -clock CLK [get_ports data_in]
set_input_delay -min 0.5 -clock CLK [get_ports data_in]
          
```

What it means?

- max: latest arrival time of input data at the chip boundary
- min: earliest arrival time (for hold analysis)

Tells when input is valid

4 Output Delay (set_output_delay)

Specifies required time for output signals relative to a clock.

```

set_output_delay -max 2.0 -clock CLK [get_ports data_out]
set_output_delay -min 0.5 -clock CLK [get_ports data_out]
          
```

What it means?

- max: latest time by which output should be valid
- min: earliest time (for hold analysis)

5 Generated Clock (create_generated_clock)

Defines a clock generated from another clock (e.g., divide, mux, PLL).

```

create_generated_clock -name CLK_DIV2 \
-source [get_ports clk] \
-divide_by 2 \
[get_pins u_div/clk_out]
          
```

What it means?

- Creates a derived clock from source clock.
- Here, clock is divided by 2 → period becomes 20 ns.

6 Virtual Clock (create_clock on non-existent port)

Defines a reference clock for I/O timing when no physical clock exists in the design.

```

create_clock -name VIRT_CLK -period 10.0
          
```

What it means?

- No physical port is associated with this clock.
- Used for specifying input/output delays w.r.t. an external interface clock.
- Common in memory interfaces (e.g., DDR), or when clock is not available in the design.

Output delay (2.0 ns max)

Specifies when output should arrive!

Necessary for all internally generated clocks.

Always define generated clocks to avoid unconstrained paths!

part of the design.

Used clock the

Key Takeaways:

- Start with primary clocks.
- Define all generated and virtual clocks.
- Define uncertainty to model real-world variations.
- Verify using reports (report_clocks, check_timing).
- Set input/output delays for all interface signals.

Right Constraints
Build a Reliable

3/5

Add Design Rule and Interface Constraints in SDC

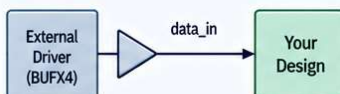
Model the real design environment for accurate synthesis and timing analysis

Good C
Better Imp

1 Driving Cell (set_driving_cell)

Specify the reference driving cell for input ports.

```
set_driving_cell -lib_cell BUF4 \
[get_ports data_in]
```

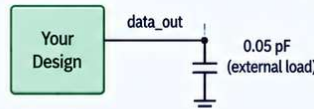


- Helps the tool model realistic input transition.
- Affects input slew, cell sizing and timing.
- Use a valid cell from the technology library.

2 Output Load (set_load)

Specify the expected external load on output ports.

```
set_load 0.05 [get_ports data_out]
```

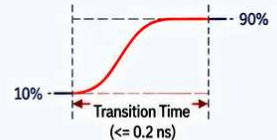


- Models the real load seen by the output.
- Important for accurate delay and transition.
- Use value provided by the interface/board specs.

3 Max Transition (set_max_transition)

Specify the maximum allowed transition time.

```
set_max_transition 0.2 [get_ports *]
```

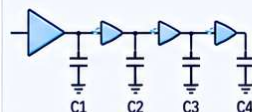


- Prevents slow slews which degrade timing.
- Helps maintain signal integrity.
- Can be applied on ports, nets or cells.

4 Max Capacitance (set_max_capacitance)

Specify the maximum load capacitance.

```
set_max_capacitance 0.1 [get_ports *]
```

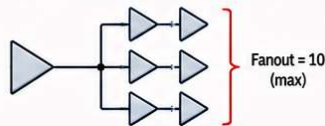


- Prevents excessive capacitive loading.
- Controls RC delay and improves timing.
- Use value based on technology and board specs.

5 Max Fanout (set_max_fanout)

Specify the maximum number of loads a net can drive.

```
set_max_fanout 10 [get_ports data_out]
```



- Controls buffering and net loading.
- Avoids high fanout which can degrade timing.
- Apply on ports, nets or cells.

6 Design Rule Constraints (DRC)

Set limits for better implementation quality.

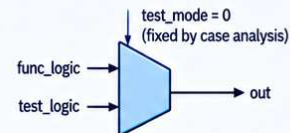
```
set_max_transition 0.2 [get_ports *]
set_max_capacitance 0.1 [get_ports *]
set_max_fanout 10 [get_ports *]
```

- These help control:
- ✓ Transition / Slew
 - ✓ Capacitive loading
 - ✓ Fanout count
 - ✓ Signal integrity
 - ✓ Better QoR and timing closure

7 Case Analysis (set_case_analysis)

Set a constant value on signals (e.g., mode select).

```
set_case_analysis 0 [get_ports test_mode]
```



- Helps the tool analyze the correct mode.
- Reduces false paths and unreachable logic.
- Common for test mode, power mode, etc.

8 Clock Groups (set_clock_groups)

Define asynchronous clock domains.

```
set_clock_groups -asynchronous \
-group {CLK_A} \
-group {CLK_B}
```



- Prevents false timing between unrelated clocks.
- Used for asynchronous or independent clocks.
- Can also use -exclusive or -physically_separated.

Key Takeaways:

- ✓ Model the real environment using driving cells and loads.
- ✓ Apply design rule constraints (transition, capacitance, fanout).
- ✓ Use case analysis for mode-specific analysis.
- ✓ Use clock groups to avoid false paths between unrelated clocks.

Accurate Con
Lead to Bett

SDC File Example – Part 1/3 (Clocks and Basic Constraints)

This part defines primary clocks, clock waveform, generated clocks, virtual clocks and clock related settings.

Good C
Lead to Ti

top.sdc (Part 1/3)

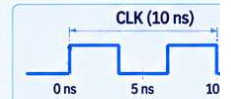
```
1 # =====
2 # top.sdc - Part 1/3 : Clocks and Basic Constraints
3 # This file contains clock definitions and clock related constraints
4 # =====
5
6 #-----
7 # 1. Primary Clock
8 #-----
9 create_clock -name CLK -period 10.0 -waveform {0 5} [get_ports clk]
10
11 # Optional: Another primary clock (e.g., for a different domain)
12 create_clock -name AUX_CLK -period 20.0 -waveform {0 10} [get_ports aux_clk]
13
14 #-----
15 # 2. Generated Clock (divide by 2 example)
16 #-----
17 create_generated_clock -name CLK_DIV2 \
18   -source [get_ports clk] \
19   -divide_by 2 \
20   [get_pins u_div/clk_out]
21
22 #-----
23 # 3. Virtual Clock (for I/O timing)
24 #-----
25 create_clock -name VIRT_CLK -period 10.0
26
27 #-----
28 # 4. Clock Uncertainty
29 #-----
30 set_clock_uncertainty -setup 0.1 [get_clocks CLK]
```

1 Primary Clock

Defines the main clock on a port with period and waveform.

```
create_clock -name CLK -period 10.0 -waveform {0 5} [get_ports clk]
```

- Clock name : CLK
- Period : 10 ns (100 MHz)
- Waveform : rising at 0 ns, falling at 5 ns
- Applied on port : clk

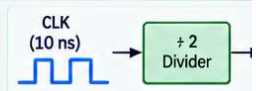


2 Generated Clock

Defines a clock generated from another clock (e.g., divide, multiply, PLL).

```
create_generated_clock -name CLK_DIV2 -source [get_ports clk] \
-divide_by 2 [get_pins u_div/clk_out]
```

- Clock name : CLK_DIV2
- Source clock : CLK
- Divide by 2 → period becomes 20 ns
- Applied on pin : u_div/clk_out



3 Virtual Clock

Defines a reference clock for I/O timing when no physical clock exists.

```
create_clock -name VIRT_CLK -period 10.0
```

```

31 set_clock_uncertainty -hold 0.05 [get_clocks CLK]
32
33 #-----
34 # 5. Clock Latency (example values)
35 #-----
36
37 set_clock_latency -source 0.2 [get_clocks CLK] # source latency
38 set_clock_latency -network 0.8 [get_clocks CLK] # network latency

```

4 Clock Uncertainty

Models clock jitter, skew and other uncertainties.

- Setup uncertainty reduces the available timing budget.
- `set_clock_uncertainty -setup 0.1 [get_clocks CLK]`
- `set_clock_uncertainty -hold 0.05 [get_clocks CLK]`

- Used for input/output delay constraints (e.g., external interface).
- No physical port is required.

Commonly used for DDR interface off-chip interfaces or when the interface is not part of the design.

5 Clock Latency

Specifies the source and network latency for the clock.

```

set_clock_latency -source 0.2 [get_clocks CLK]
set_clock_latency -network 0.8 [get_clocks CLK]

```

- Source latency : latency from clock source to the start of the clock network.
- Network latency : latency through the clock distribution network (e.g., CTS).

Key Takeaway: Correct clock definition is the foundation of accurate timing analysis.

SDC Example **P**

2/3

SDC File Example – Part 2/3 (I/O Constraints and Design Rules)

This part specifies input/output delays, driving cell, output load and design rule constraints.

Model
Real Environ
for Accurate

top.sdc (Part 2/3)

```

1 # =====
2 # top.sdc - Part 2/3 : I/O Constraints and Design Rules
3 # Specifies input/output delays, driving cells, loads and DRCs
4 # =====
5
6 # 1. Input Delay (from external device to chip)
7 # Assume data_in and ctrl_in are coming from an external device
8 set_input_delay -max 2.0 -clock CLK [get_ports {data_in ctrl_in}]
9 set_input_delay -min 0.5 -clock CLK [get_ports {data_in ctrl_in}]
10
11 # 2. Output Delay (from chip to external device)
12 # Assume data_out and status_out go to an external device
13 set_output_delay -max 2.0 -clock CLK [get_ports {data_out status_out}]
14 set_output_delay -min 0.5 -clock CLK [get_ports {data_out status_out}]
15
16 # 3. Driving Cell (model input transition)
17 # Use a library cell that represents the external driver
18 set_driving_cell -lib_cell BUF_X4 [get_ports {data_in ctrl_in}]
19
20 # 4. Output Load (model external capacitive load)
21 # Specify the expected load on output ports
22 set_load 0.05 [get_ports {data_out status_out}] # 0.05 pF
23
24 # 5. Design Rule Constraints (DRC)
25 # Set maximum transition, capacitance and fanout limits
26 set_max_transition 0.2 [get_ports *]
27 set_max_capacitance 0.1 [get_ports *]
28 set_max_fanout 10 [get_ports *]
29
30 # 6. Optional: Slew/Load on specific nets (example)
31 set_max_transition 0.15 [get_nets critical_net]
32 set_max_capacitance 0.08 [get_nets critical_net]
33
34 # =====
35 # End of Part 2/3
36 # =====

```

1 Input Delay (set_input_delay)

Specifies the arrival time of input signals relative to a clock.



Key Points:

- max : latest arrival time
- min : earliest arrival time (for hold analysis)
- Use the same clock domain as the synchronous to.

2 Output Delay (set_output_delay)

Specifies the required time for output signals relative to a clock.

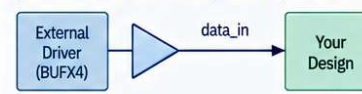


Key Points:

- max : latest time by which output must arrive
- min : earliest time (for hold analysis)
- Models the external device timing

3 Driving Cell (set_driving_cell)

Specifies the reference driving cell for input ports.



Key Points:

- Models the input transition realistically
- Use a valid library cell from your target technology library.
- Helps in accurate slew, delay and fanout analysis

4 Output Load (set_load)

Specifies the expected external load on output ports.



Key Points:

- Models the real capacitive load seen by the output
- Used for accurate delay and transition analysis
- Value comes from the interface/board specification

5 Design Rule Constraints (DRC)

Set limits on transition, capacitance and fanout.

```

set_max_transition 0.2 [get_ports *]
set_max_capacitance 0.1 [get_ports *]
set_max_fanout 10 [get_ports *]

```

Key Points:

- Prevents excessive transition and load
- Helps maintain signal integrity
- Can be applied on ports, nets, cells or the whole design

Pro Tip: Always use realistic input/output delays, driving cells and loads based on the actual interface specification. These constraints help the tool model the external environment correctly.

SDC Example **Part**

3/3

SDC File Example – Part 3/3 (Exceptions, Case Analysis and Advanced Constraints)

This part includes false paths, multicycle paths, clock groups, case analysis, propagated clocks and other useful constraints.

Clean C
Reliable

top.sdc (Part 3/3)

```

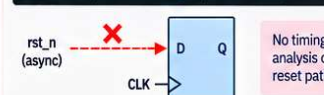
1 # =====
2 # top.sdc - Part 3/3 : Exceptions, Case Analysis and Advanced
3 # =====
4
5 # 1. False Path (asynchronous reset)
6 set_false_path -from [get_ports rst_n]
7
8 # 2. Multicycle Path (example: 2-cycle path)
9 set_multicycle_path 2 -setup -from [get_cells u_reg1] -to [get_cells u_reg2]
10 set_multicycle_path 1 -hold -from [get_cells u_reg1] -to [get_cells u_reg2]
11
12 # 3. Clock Groups (asynchronous clock domains)
13 set_clock_groups -asynchronous \
14   -group {CLK} \
15   -group {AUX_CLK} \
16   -group {VIRT_CLK}
17
18 # 4. Case Analysis (functional mode)
19 set_case_analysis 0 [get_ports test_mode] # 0 = functional mode
20
21 # 5. Propagated Clock (after CTS)
22 set_propagated_clock [all_clocks]
23
24 # 6. Clock Transition (optional)
25 set_clock_transition 0.1 [get_clocks CLK]

```

1 False Path

Used to ignore timing on paths that are not functionally timed (e.g., asynchronous reset).

```
set_false_path -from [get_ports rst_n]
```



No timing analysis on reset path.

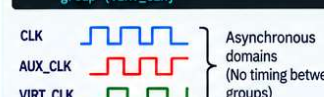
3 Clock Groups

Declares asynchronous or exclusive clock domains.

```

set_clock_groups -asynchronous \
  -group {CLK} \
  -group {AUX_CLK} \
  -group {VIRT_CLK}

```



5 Propagated Clock

Used after CTS when the clock tree is built. The tool uses real clock latency and transition from the routed clock tree.

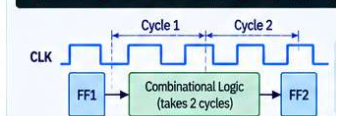
2 Multicycle Path

Allows more than one clock cycle for a path.

```

set_multicycle_path 2 -setup -from [get_cells u_reg1] -to [get_cells u_reg2]
set_multicycle_path 1 -hold -from [get_cells u_reg1] -to [get_cells u_reg2]

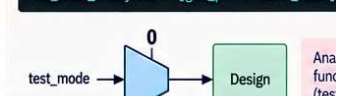
```



4 Case Analysis

Sets a constant value on a signal (e.g., mode select).

```
set_case_analysis 0 [get_ports test_mode]
```



6 Clock Transition / Latency

Specifies clock transition and latency (optional or postponed).

```
26
27 # 7. Clock Slew/Latency Update (post-CTS example)
28 set_clock_latency -source 0.2 [get_clocks CLK]
29 set_clock_latency -network 0.8 [get_clocks CLK]
30
31 # 8. Optional: Disable timing on specific paths (example)
32 set_disable_timing [get_cells u_iso/EN]
33
34 # 9. Optional: Define operating conditions (if needed)
35 set_operating_conditions -max SS -min FF
36 # End of SDC
37
38 # =====
```

real clock latency and transition from the routed clock net.

`set_propagated_clock [all_clocks]`

Before CTS: Ideal clock (no insertion delay)
After CTS : Propagated clock (real insertion, skew, transition)

7 Disable Timing (optional)

Disables timing on specific pins/cells (e.g., isolation, test logic).

`set_disable_timing [get_cells u_iso/EN]`

`set_clock_transition 0.1 [get_clocks CLK]`
`set_clock_latency -source 0.2 [get_clocks CLK]`
`set_clock_latency -network 0.8 [get_clocks CLK]`

8 Operating Conditions (optional)

Specifies PVT corners if not set in the tool flow.

`set_operating_conditions -max SS -min FF`

**Key Takeaways:**

- ✔ Use false paths for non-functional paths (e.g., reset).
- ✔ Use multicycle paths where applicable.
- ✔ Define clock groups for asynchronous clocks.

- ✔ Set case analysis for correct mode.
- ✔ Use propagated clocks after CTS.
- ✔ Add other constraints as per design requirements.

Well Constrained Design Leads to Successful Results